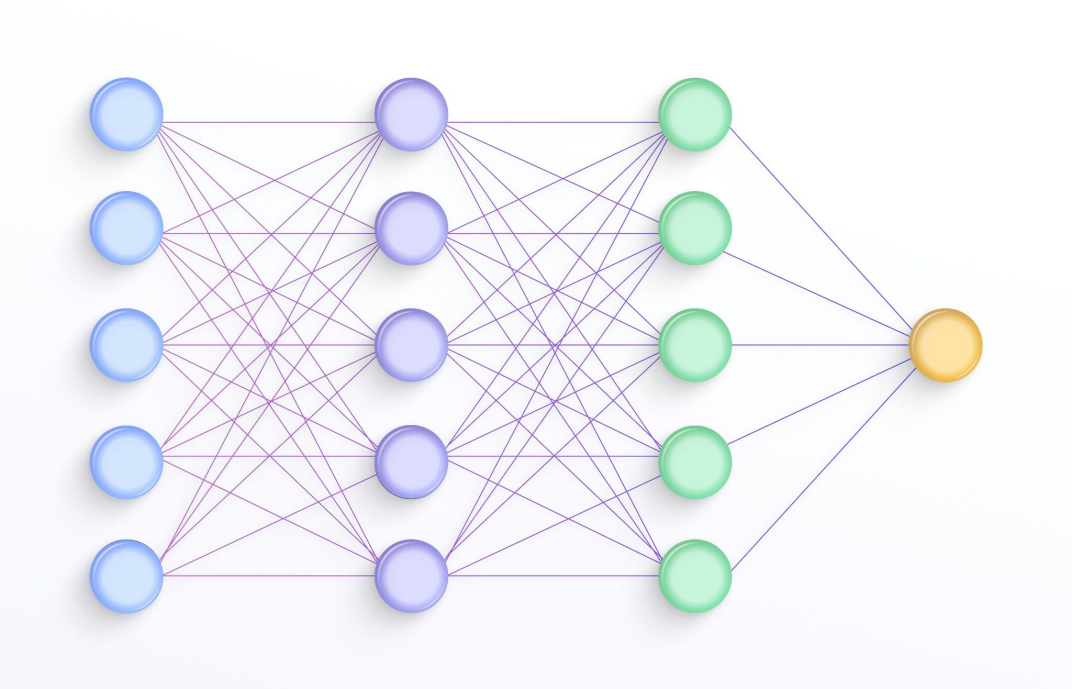
# Introduction to Neural Networks (NN)

**Rui Cao**
Assistant Professor
Information Systems Technology and Design

rui_cao@sutd.edu.sg
https://abril4416.github.io/

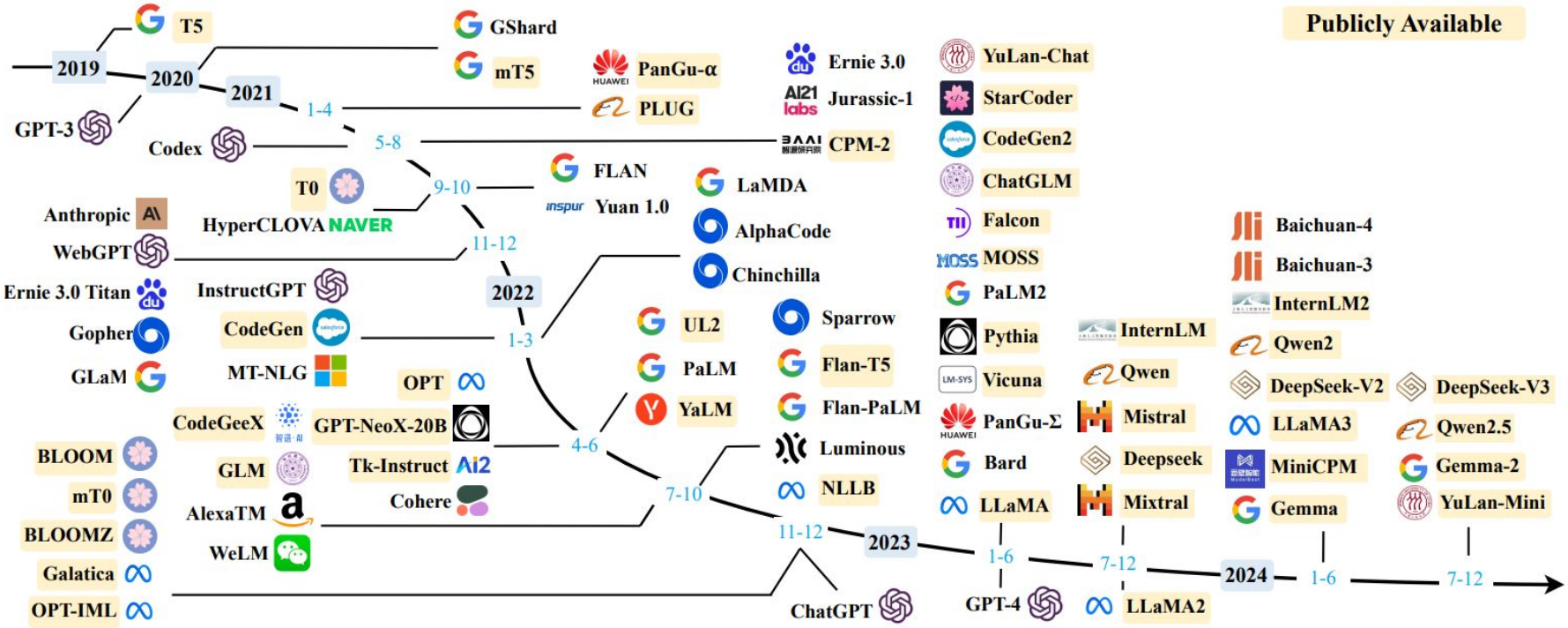


Image

SINGAPORE UNIVERSITY OF TECHNOLOGY AND DESIGN

# Examples of Deep Neural Networks

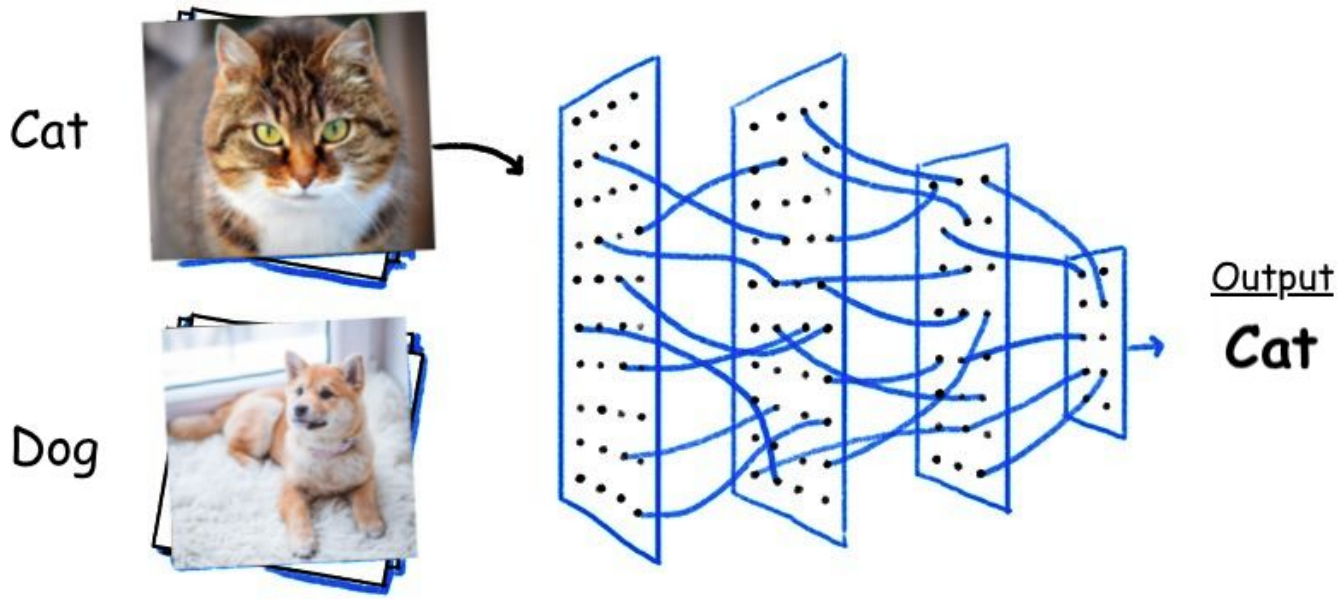
- Large language models: from 2019




paper

# Examples of Deep Neural Networks

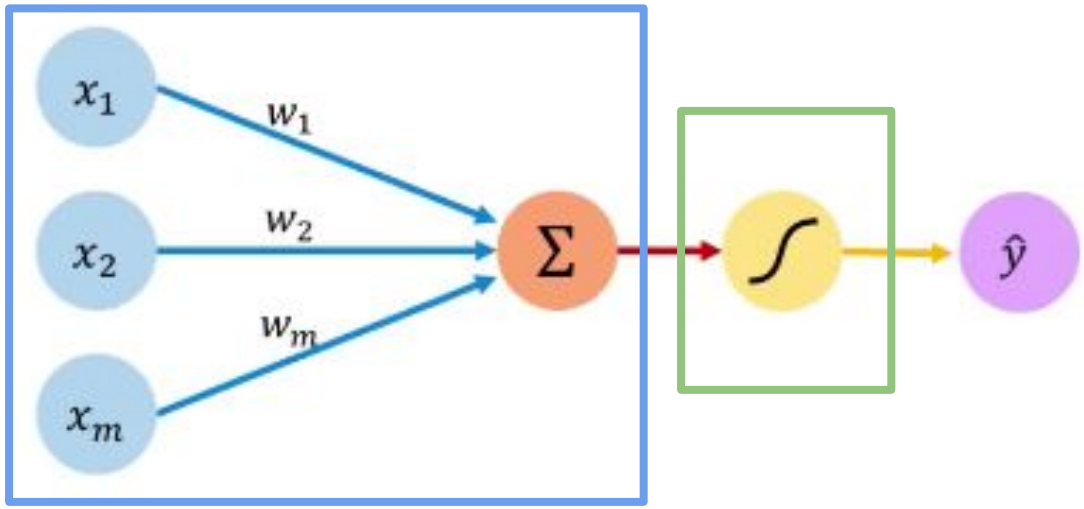
- Image classification models: from 2012
  - E.g., AlexNet (8), ResNet (152)…




image

# Start from Simple

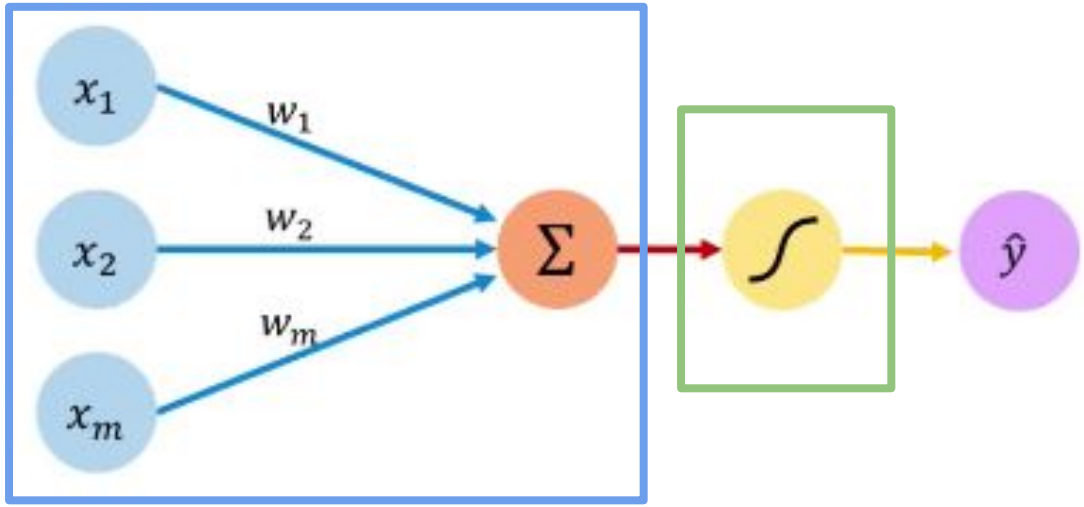
- Artificial neurons: composition of NN
  - Linear transformation of input
  - Non-linear transformation with activation functions




wiki

# Logistic Regression

- A single neuron
  - Activation function: Sigmoid function




$$\hat{y} = \sigma(w^T x + b)$$

# Outline Today

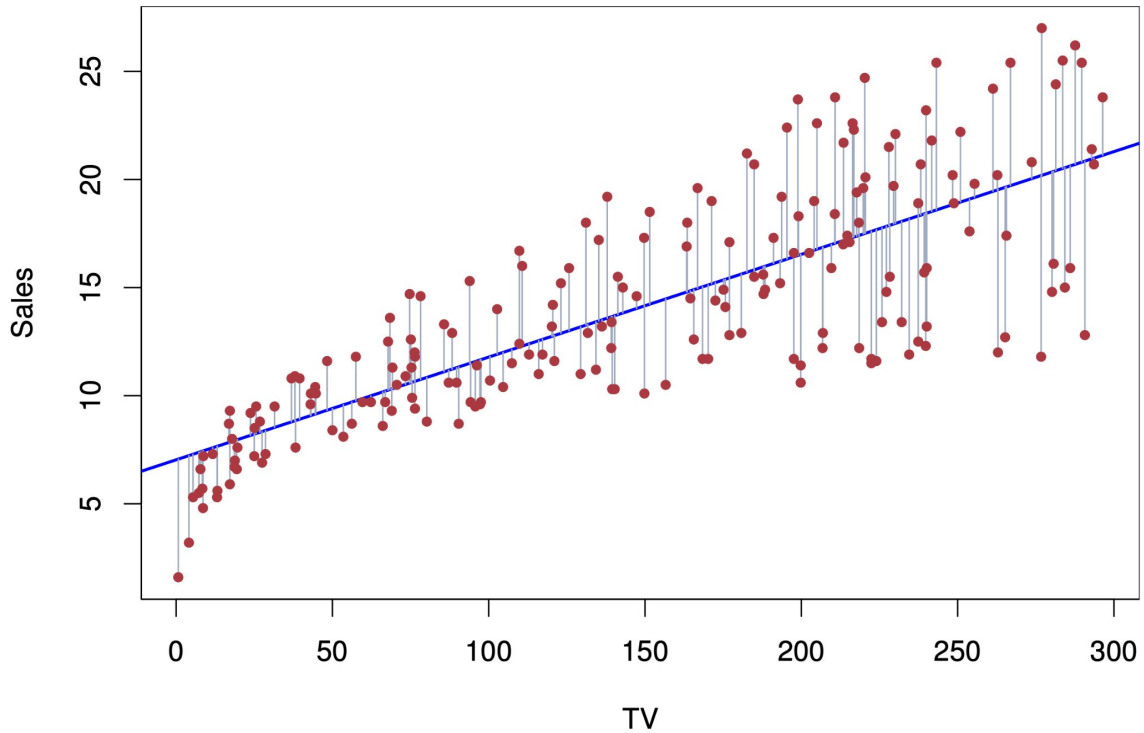
- Simple: A Single Neuron
  - Recall: logistic regression
  - Loss function
  - Learn your model: gradient descent
- Complex: Neural Networks
  - Multilayer Perceptron
  - Activation functions and loss functions
  - Training: gradient backpropagation
  - Optimization and hyperparameters

# Outline Today

- **Simple:** A Single Neuron
  - Recall: logistic regression
  - Loss function
  - Learn your model: gradient descent
- Complex: Neural Networks
  - Multilayer Perceptron
  - Activation functions and loss functions
  - Training: gradient backpropagation
  - Optimization and hyperparameters

# Recall: Logistic Regression

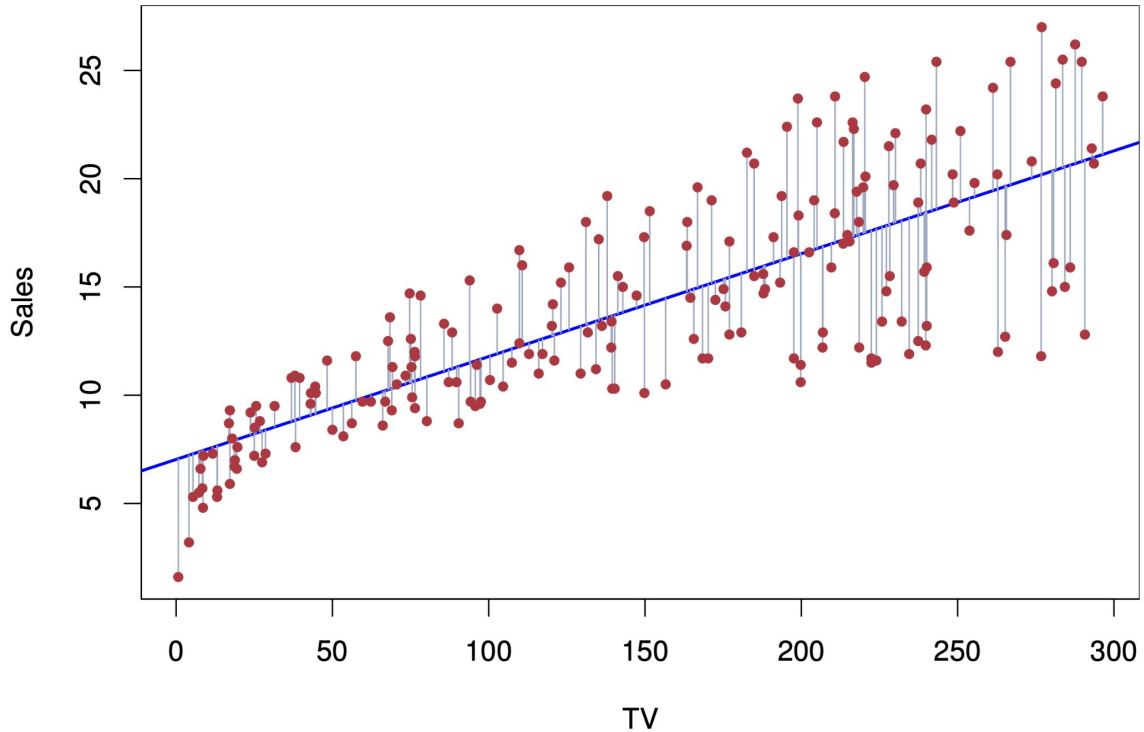
- Binary Classification:
  - Linear regression $\hat{y} = w^T x + b$




image

# Recall: Logistic Regression

- Binary Classification:
  - Linear regression $\hat{y} = w^T x + b$
  - **Why linear regression fails for classification?**



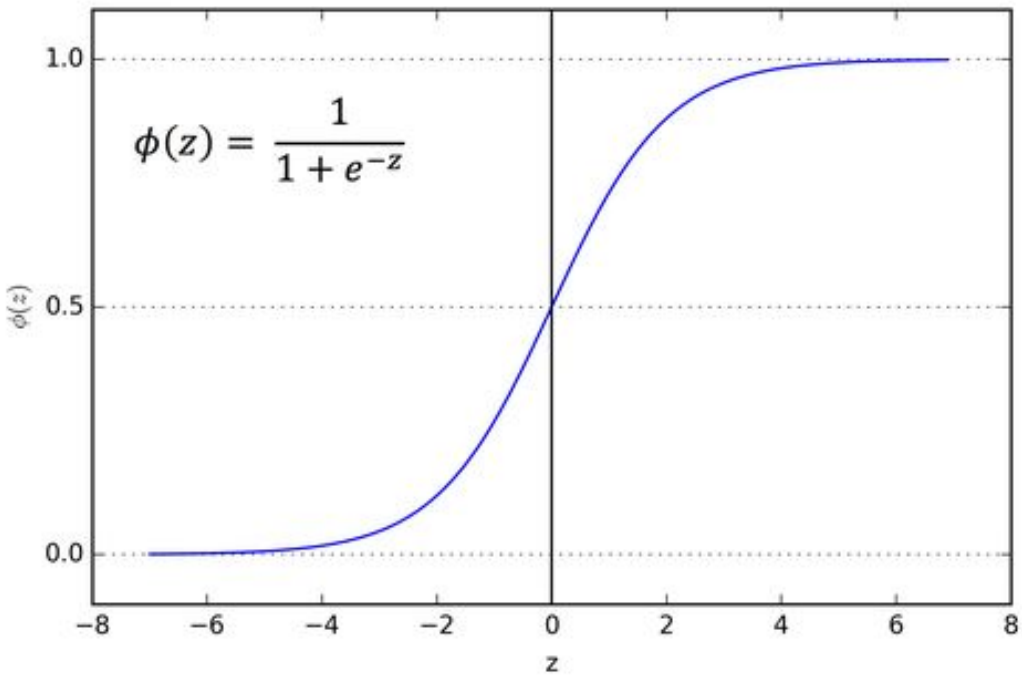

image

# Recall: Logistic Regression

- Map the linear output to probability with <u>sigmoid function</u>
  - Bounded output: (0,1)

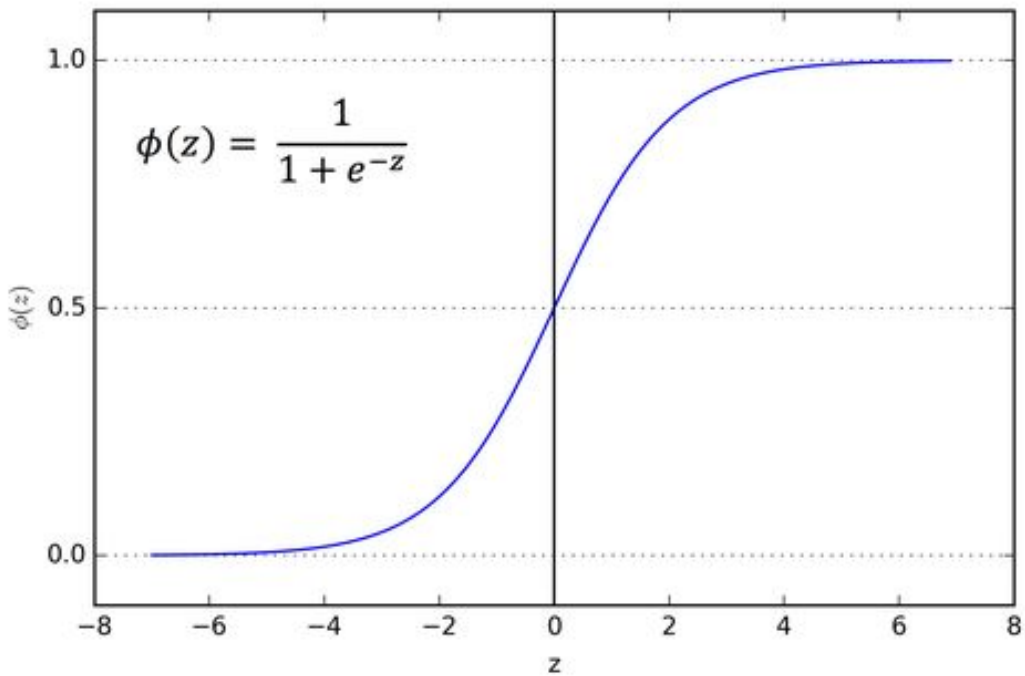
$$P(y=1|x) = \sigma(w^T x + b)$$

$$P(y=0|x) = 1 - \sigma(w^T x + b)$$



# Recall: Logistic Regression

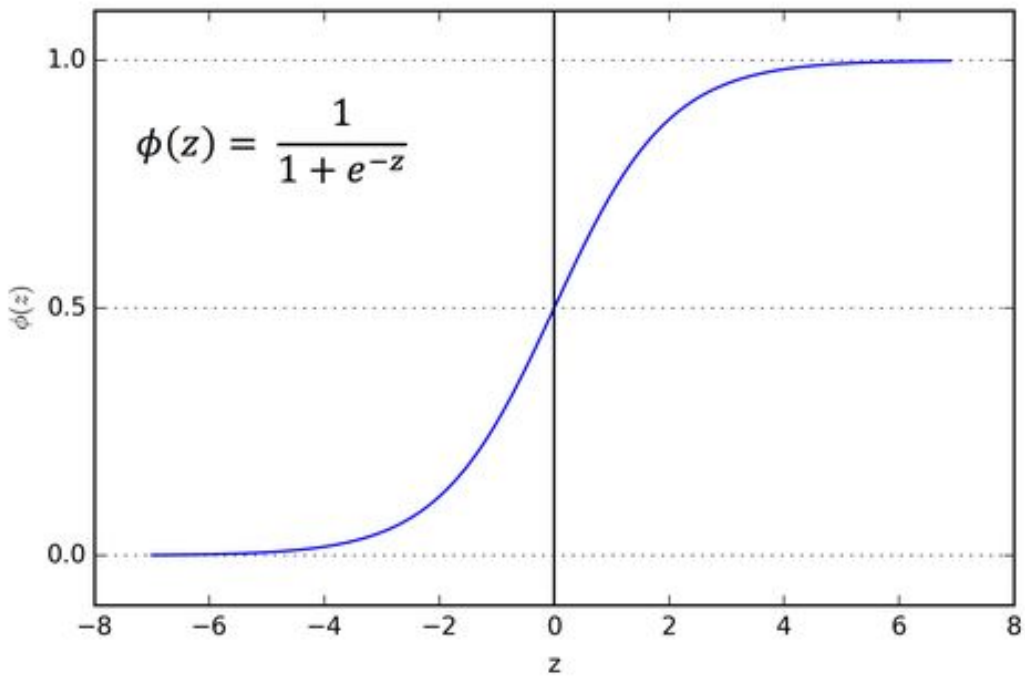
- Decision rule:
  - **Decision boundary?**




$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

# Recall: Logistic Regression

- Decision rule:
  - **Decision boundary?** $w^T x + b = 0$
  - Linear boundary but with a probabilistic output




$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

# Model Learning: Maximum Likelihood

- Likelihood for classification:
  - A classification model $P(y \mid x; \theta)$, write as $p_\theta(y \mid x)$
  - Equation: $\mathcal{L}(\theta) = \prod_{i=1}^{N} p_\theta(y_i \mid x_i)$
  - For simplicity, log-likelihood: $\ell(\theta) = \sum_{i=1}^{N} \log p_\theta(y_i \mid x_i)$
  - Learning algorithm: maximum likelihood estimation **(MLE)**

# Model Learning: Maximum Likelihood

- i.i.d Assumption: independent and identically distributed
  - Each data point does not influence another
    - $P(x_1, x_2, ..., x_n) = \prod_{i=1}^{n} P(x_i)$
  - All samples come from the same probability distribution
    - $x_i \sim P(x)$
  - Core assumption in ML and most deep learning methods

# Logistic Regression

- Recall probability for an example:

$$P(y = 1 \mid x) = \sigma(z) = \sigma(w^T x)$$

$$P(y = 0 \mid x) = 1 - \sigma(w^T x)$$

  - y is either 0 or 1: $P(y|x) = \sigma(z)^y (1 - \sigma(z))^{1-y}$

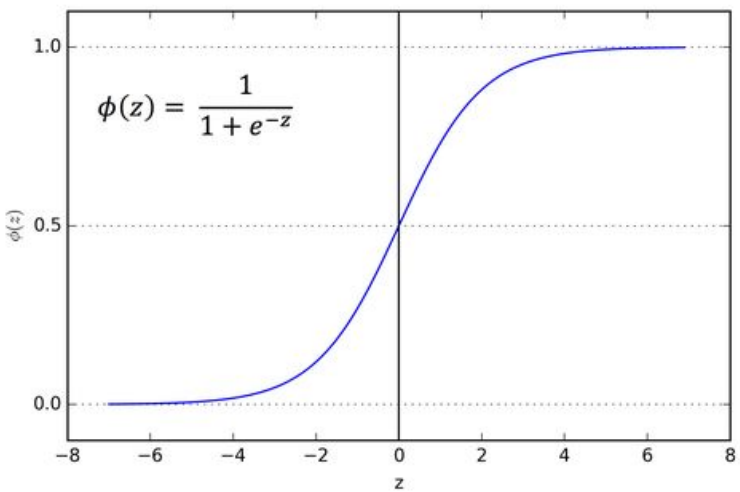
- log-likelihood: N samples

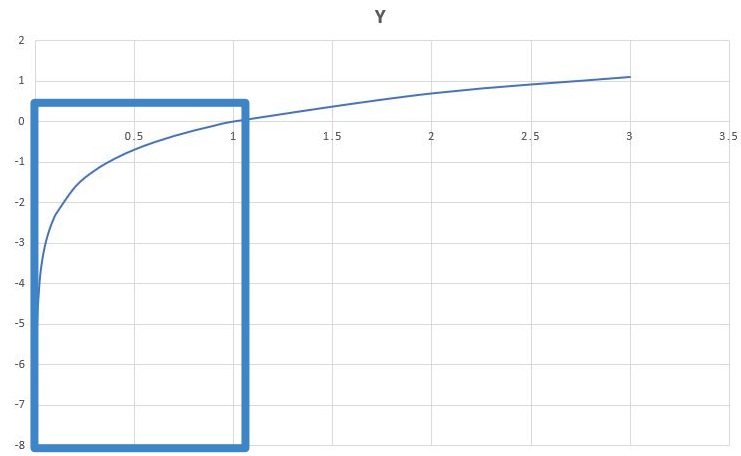
$$\ell(w) = \sum_{i=1}^{N} \left[ y_i \log \sigma(w^T x_i) + (1 - y_i) \log(1 - \sigma(w^T x_i)) \right]$$

  - Learning objective: **<u>maximize log-likelihood</u>**
    - Minimize $-l(w)$

# Loss Function

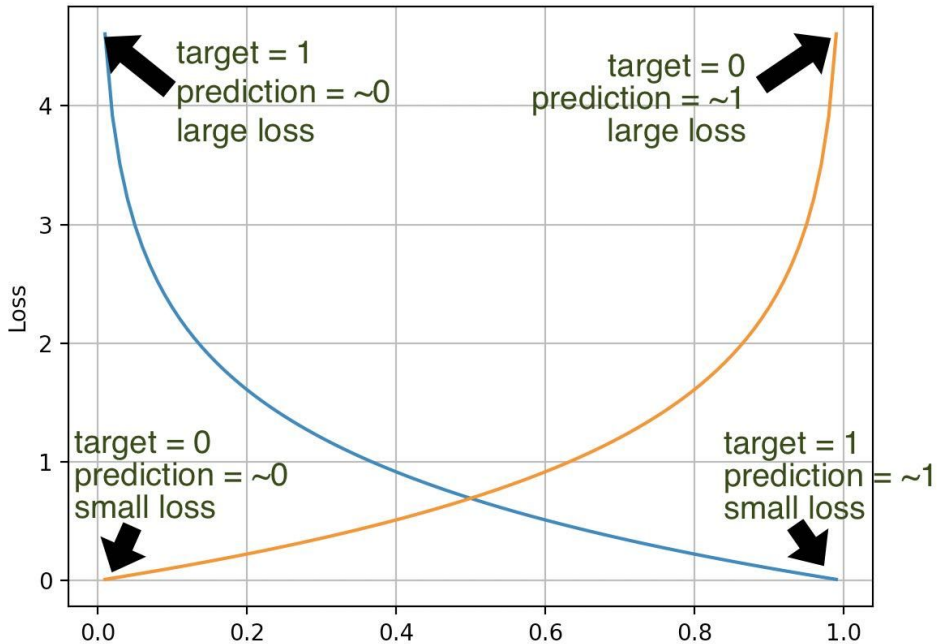
- Behavior of the loss function
  - One example: $\boxed{-\left[y\log(\sigma(w^\top x))\right.}+(1-y)\log(1-\sigma(w^\top x))]$
  - If **y=1**, we want $\log(\sigma(w^\top x))$ to be close to 1
    - In other words, $w^\top x + b$ >>0



# Loss Function

- Behavior of the loss function, we want
  - Linear transformation<<0, when y=0
  - Linear transformation>>0, when y=1

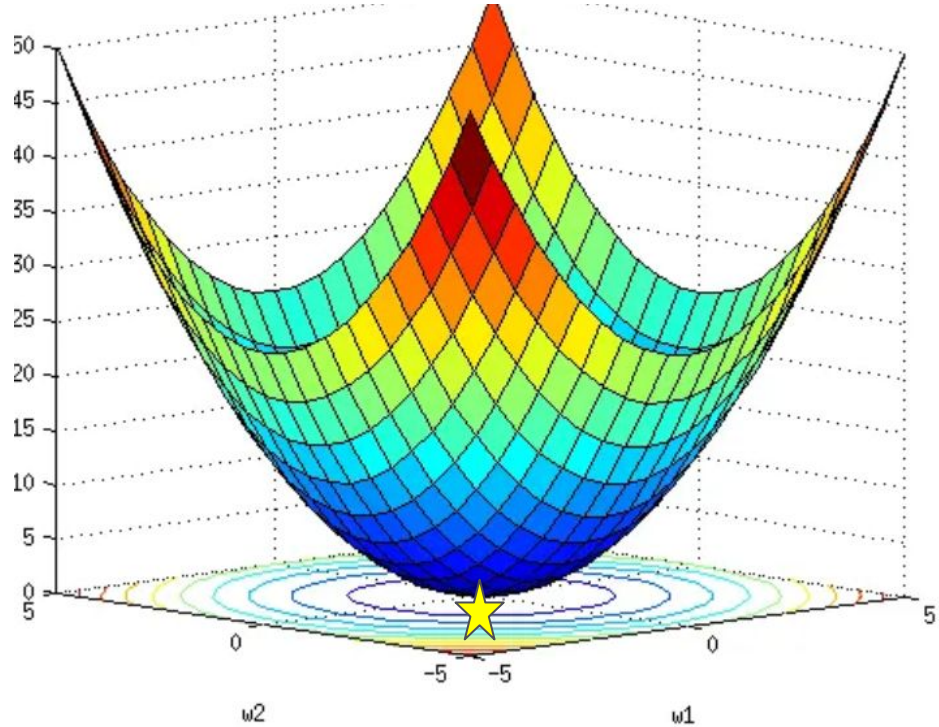


ANY QUESTION

# Learn Your Model

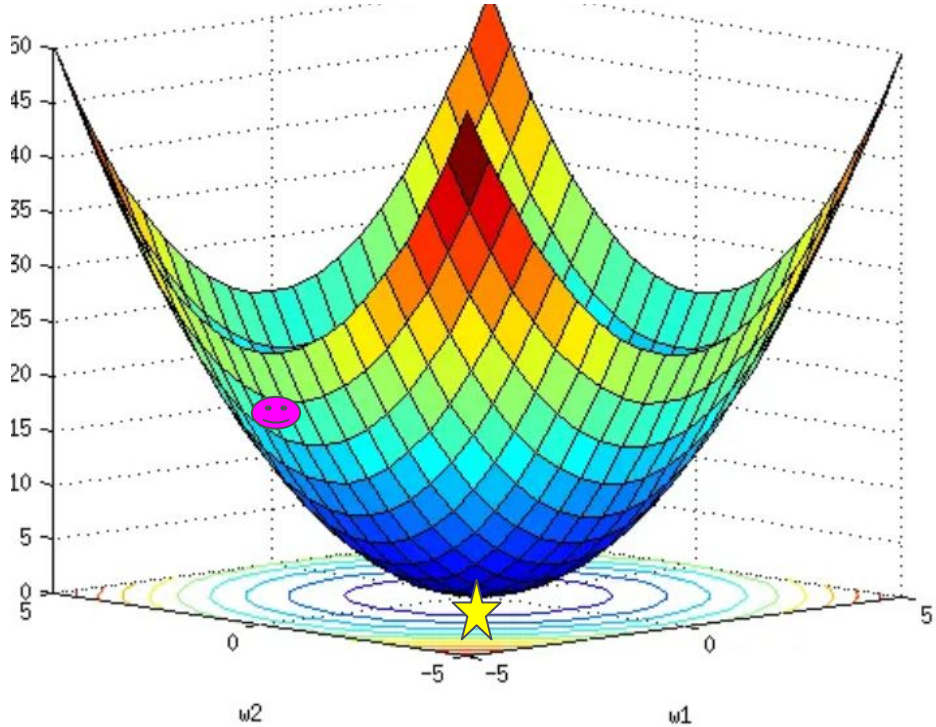
- Plot of the loss function J (-log-likelihood) with respects to weights
  - Assume 2 input features
  - Goal: lowest point

$$\hat{y} = \frac{1}{1 + \exp\big(-(w_1 x_1 + w_2 x_2 + b)\big)}$$



# Gradient Descent

- Random initialize your weights, starting point in loss
- Update weights iteratively
  - Take small steps in the direction of **steepest decrease** of the loss



# Gradient Descent

- Gradient of the loss: **the direction of ?**

# Gradient Descent

- Gradient of the loss: the direction of **<u>steepest increase</u>**
- Denotations:
  - Linear transformation and outputs

$$z = w^T x + b \qquad \hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

  - Loss (one example for simplicity)

$$J = -\left[y \log \hat{y} + (1-y)\log(1-\hat{y})\right]$$

- Minimize the loss: opposite direction
  - Update rule for each iteration (η, the step length, learning rate)

$$w \leftarrow w - \eta \frac{\partial J}{\partial w}$$

# Gradient Descent

- Gradient of the loss: the direction of **<u>steepest increase</u>**
- Denotations:
  - Linear transformation and outputs

$$z = w^T x + b \qquad \hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

  - Loss (one example for simplicity)

$$J = -\left[y \log \hat{y} + (1-y)\log(1-\hat{y})\right]$$

- Minimize the loss: opposite direction
  - Update rule for each iteration

$$w \leftarrow w - \eta \frac{\partial J}{\partial w} \qquad b^{new} = b^{old} - \eta \frac{\partial J}{\partial b}$$

# Gradient Descent: Weights

- Derivative of J, w.r.t weights: **chain rule**

$$J = -\left[y \log \hat{y} + (1-y)\log(1-\hat{y})\right]$$

$$z = w^T x + b \qquad \hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

$$w \leftarrow w - \eta \frac{\partial J}{\partial w}$$

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$$\frac{\partial J}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \qquad \frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y}) \qquad \frac{\partial z}{\partial w} = x$$

# Gradient Descent: Weights

- Derivative of J, w.r.t weights: **chain rule**

$$J = -\left[y \log \hat{y} + (1-y)\log(1-\hat{y})\right]$$

$$z = w^T x + b \qquad \hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

$$w \leftarrow w - \eta \frac{\partial J}{\partial w}$$

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$$\frac{\partial J}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \qquad \frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y}) \qquad \frac{\partial z}{\partial w} = x$$

$$\frac{\partial J}{\partial w} = (\hat{y} - y)x \qquad w \leftarrow w - \eta(\hat{y} - y)x$$

# Gradient Descent: Bias

- Derivative of J w.r.t weights: **chain rule**

$$J = -\left[y \log \hat{y} + (1-y)\log(1-\hat{y})\right]$$

$$z = w^T x + b \qquad \hat{y} = \sigma(z) \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

$$b^{new} = b^{old} - \eta \frac{\partial J}{\partial b}$$

- Similarly

$$\frac{\partial J}{\partial b} = \hat{y} - y \qquad b \leftarrow b - \eta(\hat{y} - y)$$

# Gradient Descent: Batch

- Extend to multiple examples (n examples)

$$\frac{\partial J}{\partial w} = (\hat{y} - y)x \qquad \frac{\partial J}{\partial b} = \hat{y} - y$$

$$\frac{\partial J}{\partial w} = \sum_{i=1}^{n} (\hat{y}_i - y_i)x_i \qquad \frac{\partial J}{\partial b} = \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

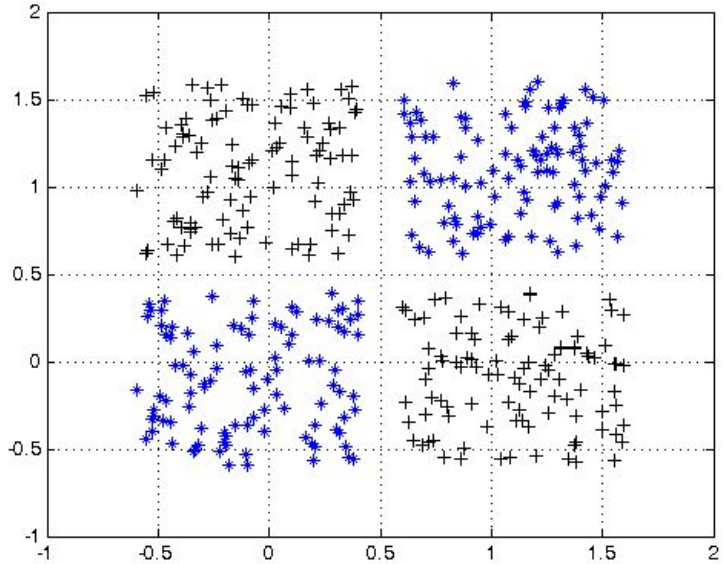
$$w \leftarrow w - \eta \sum_{i} (\hat{y}_i - y_i)x_i \qquad b \leftarrow b - \eta \sum_{i} (\hat{y}_i - y_i)$$

# High-level Idea of Model Learning

- Initialize weights and bias (parameters)
- Loop until convergence, in each iteration
  - Compute gradients
  - Update weights and bias $w \leftarrow w - \eta \frac{\partial J}{\partial w}$ $\quad b^{new} = b^{old} - \eta \frac{\partial J}{\partial b}$
- Return weights

# Logistic Regression Fails

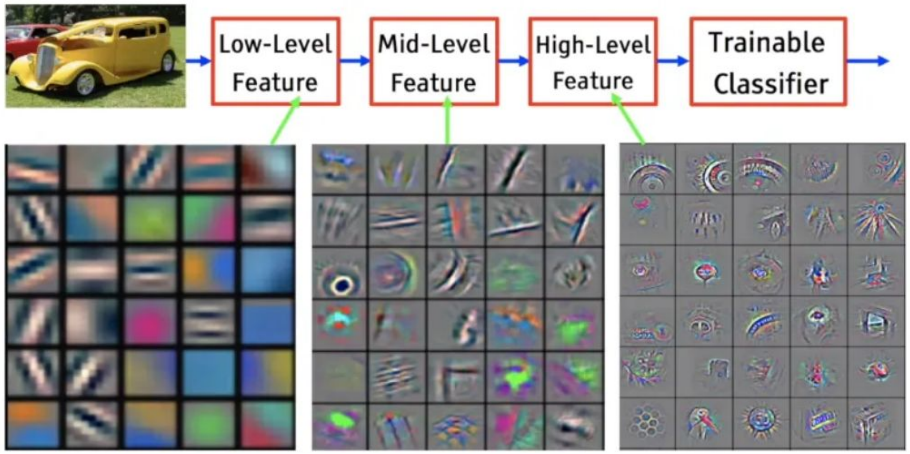
- Linear boundary $w^\top x + b = 0$
  - May not be sufficient: e.g., data points below




image

# Logistic Regression Fails

- Linear boundary $w^\top x + b = 0$
  - Real-world problems: image classification
  - Relationship between features
  - Input : image pixels
  - Relation:
    - A few form edges, two edges form a corner, several edges form a circle (hierarchical)




Image

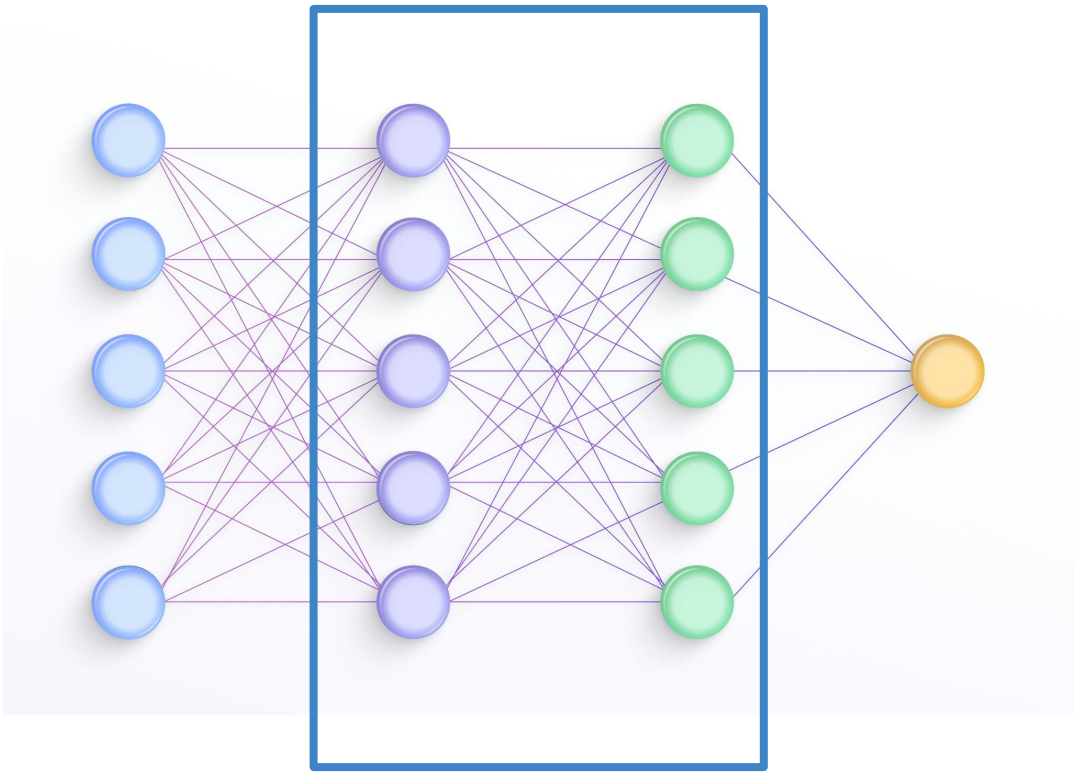
# Outline Today

- Simple: A Single Neuron
  - Recall: logistic regression
  - Loss function
  - Learn your model: gradient descent
- Complex: Neural Networks
  - Multilayer Perceptron
  - Activation functions and loss functions
  - Training: gradient backpropagation
  - Optimization and hyperparameters

ANY QUESTION

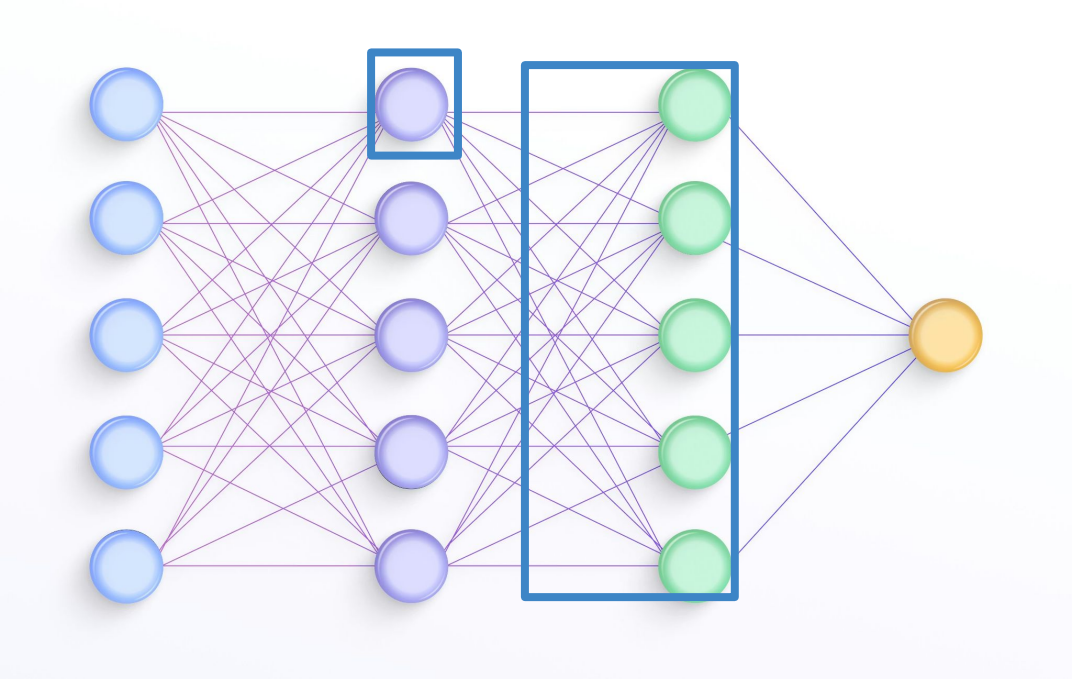
# Multilayer Perceptron (MLP)

- Multiple layers of neurons (nodes)
  - Input layer, hidden layer(s), output layer



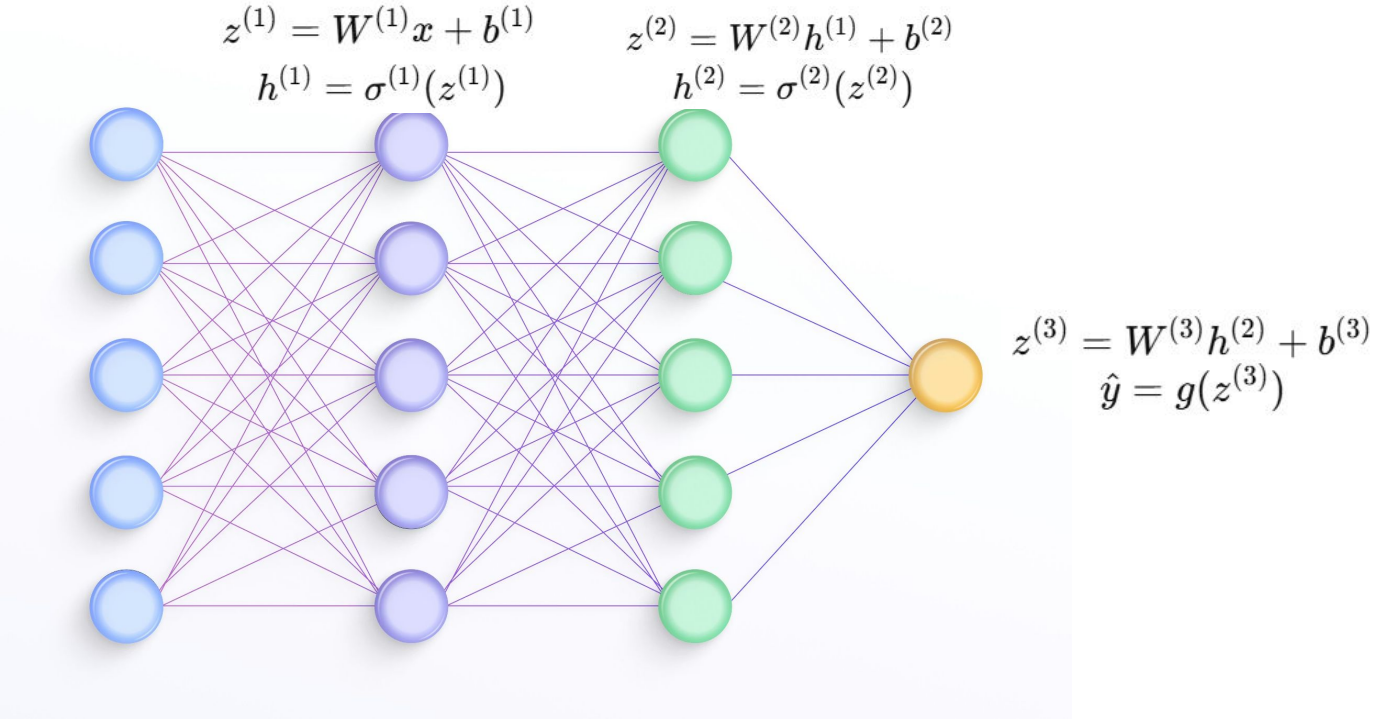
# Multilayer Perceptron (MLP)

- MLP:
  - Fully connected: Every node connects to all nodes in the next layer
  - Universal approximator: approximate almost all functions with enough neurons



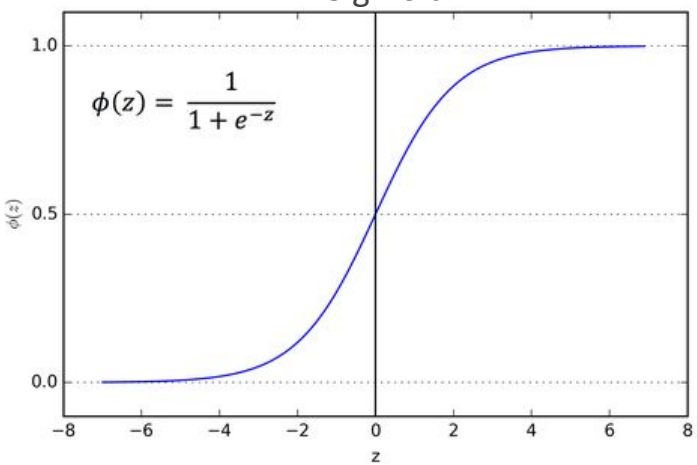
# Multilayer Perceptron (MLP)

- Nonlinear: formulations
  - Assume input $x \in \mathbb{R}^d$
  - Each layer: linear transformation + activation function

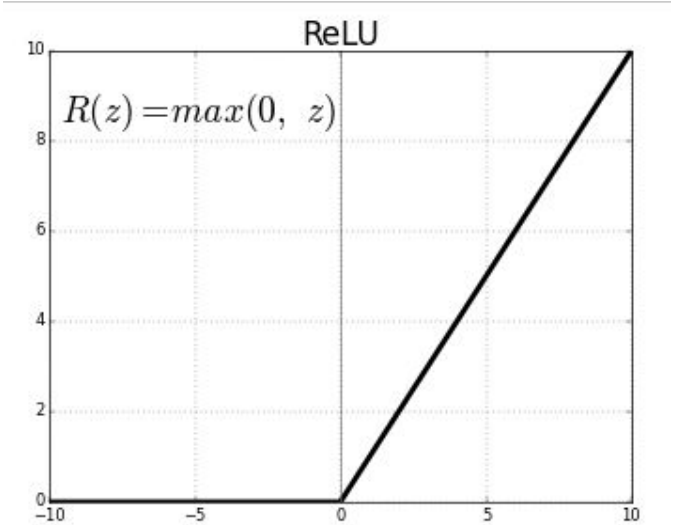


# Common Activation Functions

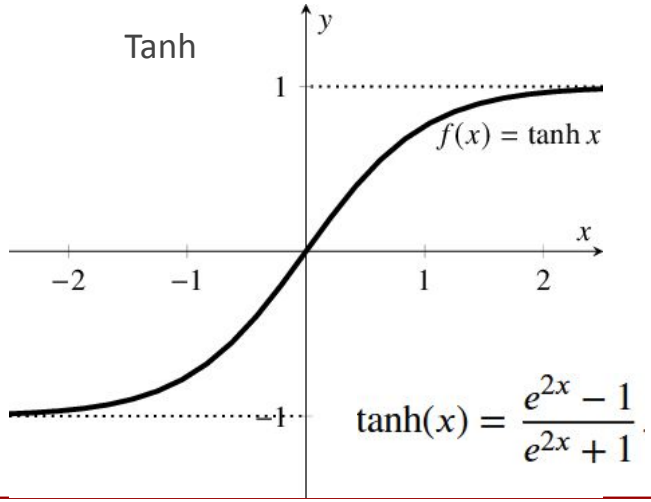
Sigmoid



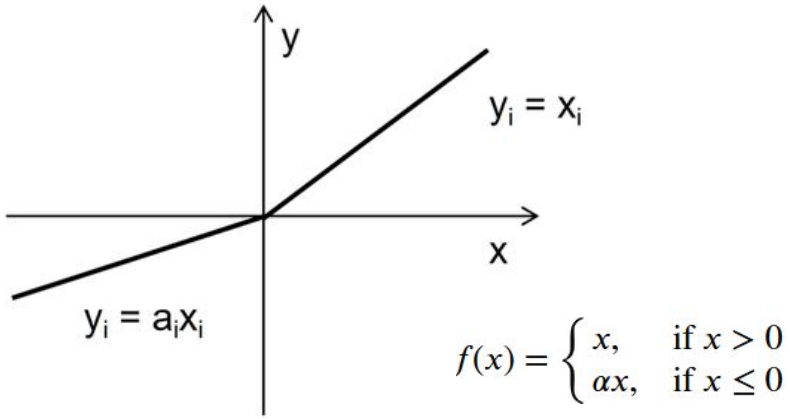

ReLU




Tanh




Leaky ReLU



# Output Layer Activation

- Depends on task (last output is z)

| Function | Identity | Sigmoid | Softmax |
|---|---|---|---|
| | $g(z) = z$ | $g(z) = \frac{1}{1+e^{-z}}$ | $\mathrm{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$ |
| Used for | Regression | Binary classification | Muti-class classification |
| Loss | MSE | Binary cross-entropy | Cross-entropy |
| | $L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$ | $L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$ | $L = -\sum_{k=1}^{C} y_k \log \hat{y}_k$ |

# Output Layer Activation

- Depends on task (last output is z)

| Function | Identity | Sigmoid | Softmax |
|---|---|---|---|
| | $g(z) = z$ | $g(z) = \frac{1}{1+e^{-z}}$ | $\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$ |
| Used for | Regression | Binary classification | Muti-class classification |
| Loss | MSE | Binary cross-entropy | Cross-entropy |
| | $L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$ | $L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$ | $L = -\sum_{k=1}^{C} y_k \log \hat{y}_k$ |

Multi-label categorical: e.g., movie classification (action, comedy, romance, science fiction … 10 types in total): **dimension of output?**

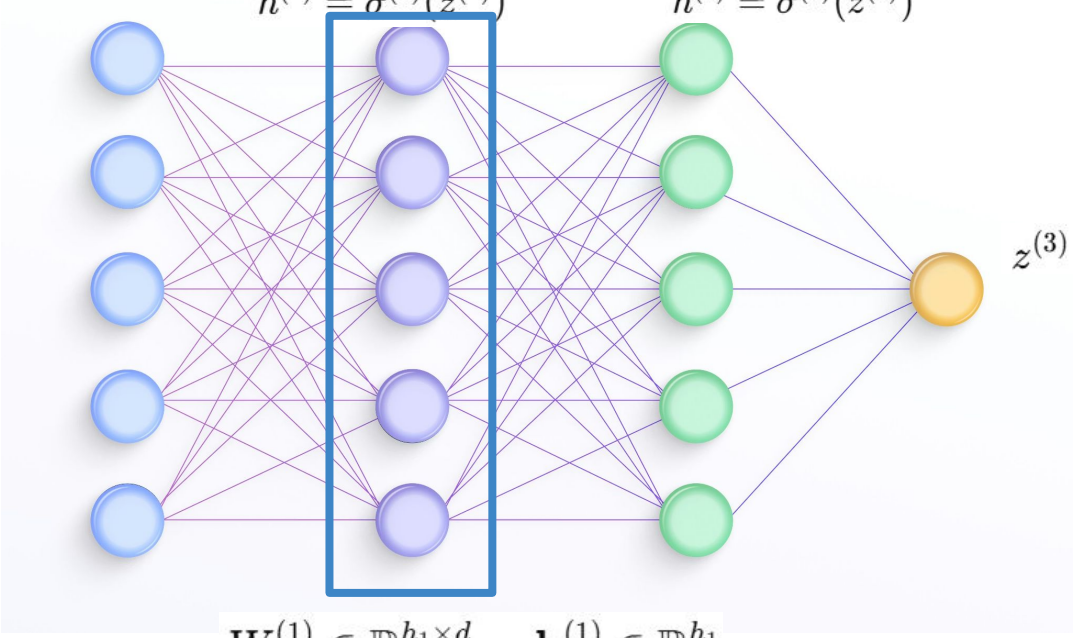
**How to design the output layer activation, what is the loss function?**

# Learning for NN

- Forward Path: for neuron j in the first hidden layer
  - Weighted sum of input features + bias $z_j^{(1)} = \sum_{i=1}^{d} w_{ji}^{(1)} x_i + b_j^{(1)}$
  - Activation function applied: $\phi\left(\sum_{i=1}^{d} w_{ji}^{(1)} x_i + b_j^{(1)}\right)$

$$z^{(1)} = W^{(1)}x + b^{(1)}$$
$$h^{(1)} = \sigma^{(1)}(z^{(1)})$$

$$z^{(2)} = W^{(2)}h^{(1)} + b^{(2)}$$
$$h^{(2)} = \sigma^{(2)}(z^{(2)})$$



$$z^{(3)} = W^{(3)}h^{(2)} + b^{(3)}$$
$$\hat{y} = g(z^{(3)})$$

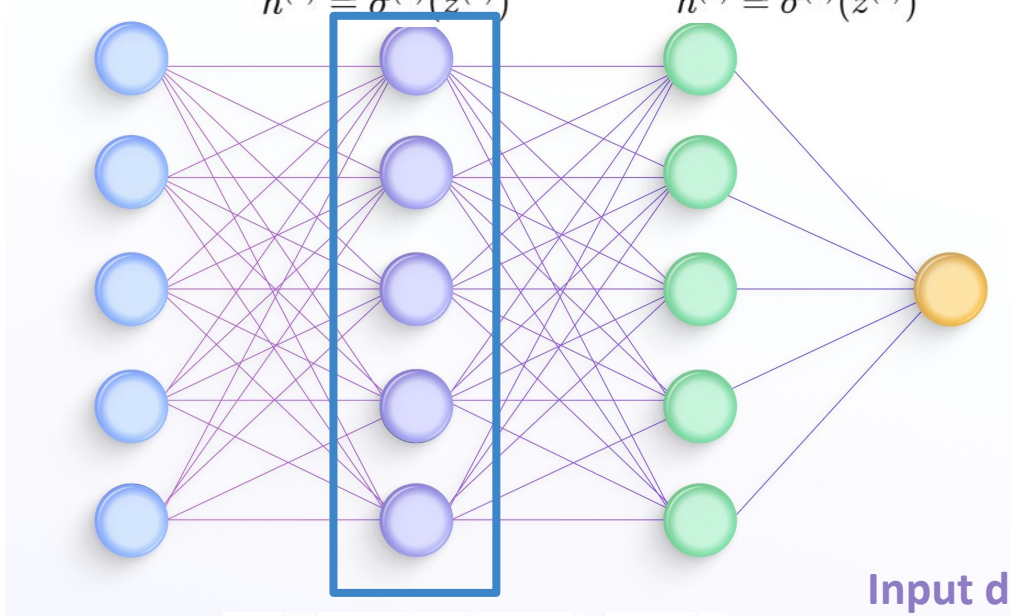
$$W^{(1)} \in \mathbb{R}^{h_1 \times d}, \quad \mathbf{b}^{(1)} \in \mathbb{R}^{h_1}$$

# Learning for NN

- Forward Path: for neuron j in the first hidden layer
  - Weighted sum of input features + bias $z_j^{(1)} = \sum_{i=1}^{d} w_{ji}^{(1)} x_i + b_j^{(1)}$
  - Activation function applied: $\phi\left(\sum_{i=1}^{d} w_{ji}^{(1)} x_i + b_j^{(1)}\right)$

$$z^{(1)} = W^{(1)}x + b^{(1)}$$
$$h^{(1)} = \sigma^{(1)}(z^{(1)})$$

$$z^{(2)} = W^{(2)}h^{(1)} + b^{(2)}$$
$$h^{(2)} = \sigma^{(2)}(z^{(2)})$$



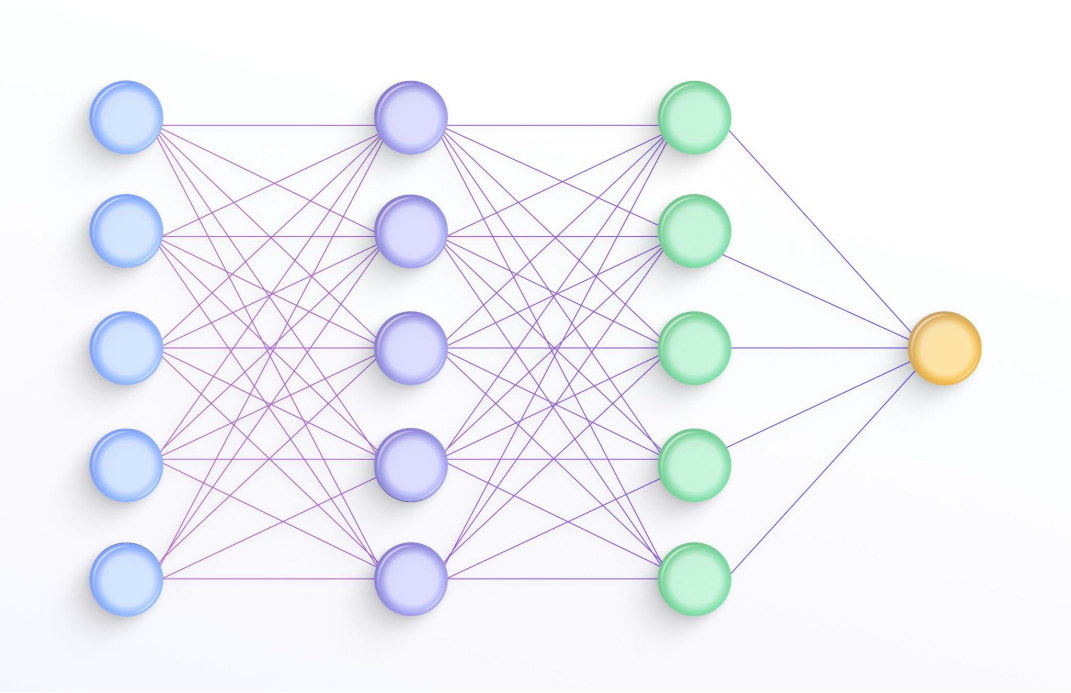
$$z^{(3)} = W^{(3)}h^{(2)} + b^{(3)}$$
$$\hat{y} = g(z^{(3)})$$

$$W^{(1)} \in \mathbb{R}^{h_1 \times d}, \quad \mathbf{b}^{(1)} \in \mathbb{R}^{h_1}$$

**Input dimension?**
**Hidden output dimension?**

# Learning for NN

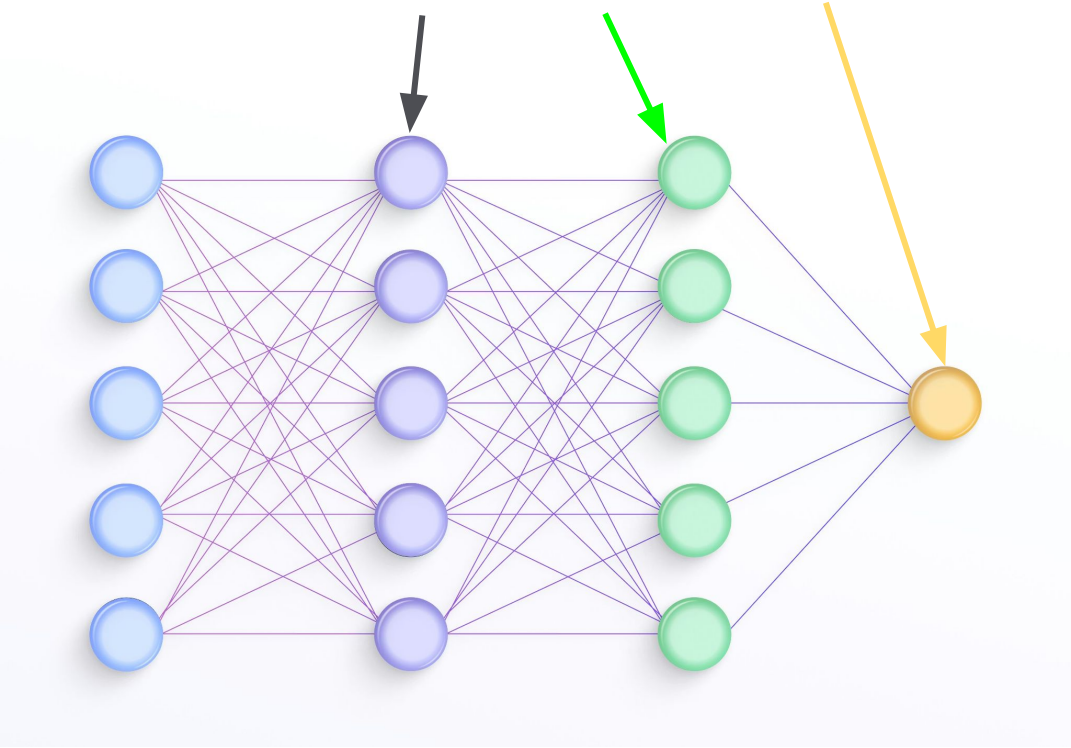
- High-level idea: gradient descent with respect to weights
  - Minimize a loss function $w \leftarrow w - \eta \frac{\partial J}{\partial w}$



# Learning for NN

- High-level idea: gradient descent with respect to weights
  - Minimize a loss function $w \leftarrow w - \eta \frac{\partial J}{\partial w}$

Weights of three layers, which to be computed first?

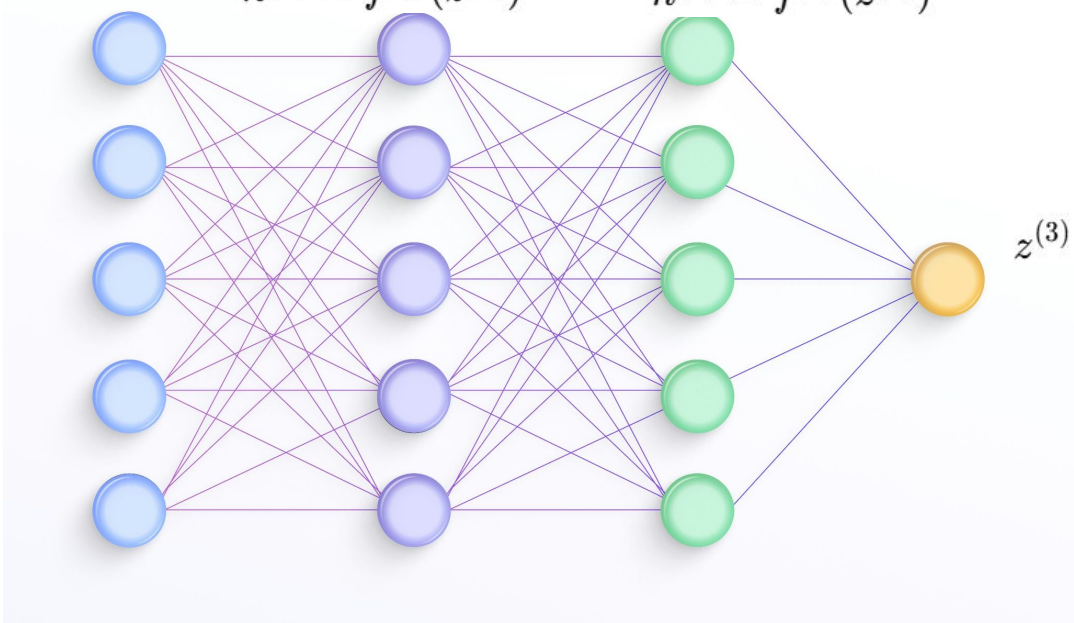


# Backpropagation

- Loss function: $J(\hat{y}, y)$
- Goal: $\frac{\partial J}{\partial w^{(l)}}, \quad \frac{\partial J}{\partial b^{(l)}}$
- Method: chain rule

$$z^{(1)} = W^{(1)}x + b^{(1)}$$
$$h^{(1)} = f^{(1)}(z^{(1)})$$

$$z^{(2)} = W^{(2)}h^{(1)} + b^{(2)}$$
$$h^{(2)} = f^{(2)}(z^{(2)})$$



$$z^{(3)} = W^{(3)}h^{(2)} + b^{(3)}$$
$$\hat{y} = g(z^{(3)})$$

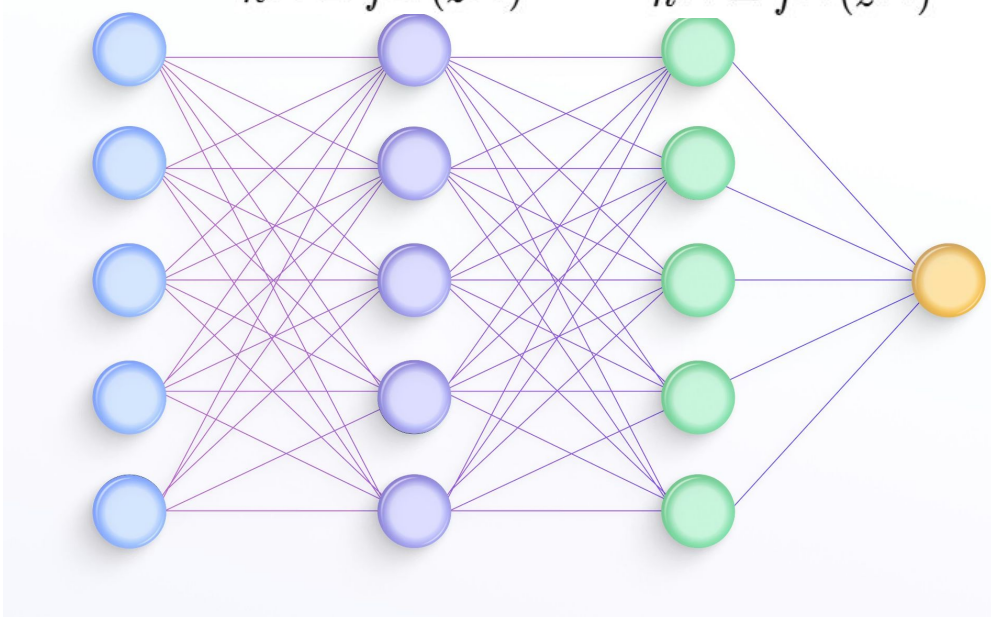
# Backpropagation

- Method: chain rule

$$\frac{\partial J}{\partial W_3} = \boxed{\frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3}} \frac{\partial z_3}{\partial W_3} \qquad \frac{\partial J}{\partial W_2} = \boxed{\frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3}} \frac{\partial z_3}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial W_2} \qquad \frac{\partial J}{\partial W_1} = \boxed{\frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3}} \boxed{\frac{\partial z_3}{\partial h_2} \frac{\partial h_2}{\partial z_2}} \frac{\partial z_2}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial W_1}$$

$$\frac{\partial J}{\partial b_3} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3} \frac{\partial z_3}{\partial b_3} \qquad \frac{\partial J}{\partial b_2} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3} \frac{\partial z_3}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial b_2} \qquad \frac{\partial J}{\partial b_1} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3} \frac{\partial z_3}{\partial h_2} \frac{\partial z_2}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial b_1}$$

$$z^{(1)} = W^{(1)}x + b^{(1)} \qquad z^{(2)} = W^{(2)}h^{(1)} + b^{(2)}$$
$$h^{(1)} = f^{(1)}(z^{(1)}) \qquad h^{(2)} = f^{(2)}(z^{(2)})$$



$$z^{(3)} = W^{(3)}h^{(2)} + b^{(3)}$$
$$\hat{y} = g(z^{(3)})$$

# Backpropagation

- Error Term: $\delta_j = \frac{\partial J}{\partial z_j}$
  - Measures how much a neuron is responsible for the final error
  - Weight matrix, error term, derivative
  - Can be re-used for gradient computation (l=2)

$$\delta^{(l)} = \left(W^{(l+1)T}\delta^{(l+1)}\right) \odot f'(z^{(l)})$$

$$\frac{\partial J}{\partial W_3} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial W_3} \qquad \frac{\partial J}{\partial W_2} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial h_2}\frac{\partial h_2}{\partial z_2}\frac{\partial z_2}{\partial W_2} \qquad \frac{\partial J}{\partial W_1} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial h_2}\frac{\partial h_2}{\partial z_2}\frac{\partial z_2}{\partial h_1}\frac{\partial h_1}{\partial z_1}\frac{\partial z_1}{\partial W_1}$$

$$\frac{\partial J}{\partial b_3} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial b_3} \qquad \frac{\partial J}{\partial b_2} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial h_2}\frac{\partial h_2}{\partial z_2}\frac{\partial z_2}{\partial b_2} \qquad \frac{\partial J}{\partial b_1} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial h_2}\frac{\partial h_2}{\partial z_2}\frac{\partial z_2}{\partial h_1}\frac{\partial h_1}{\partial z_1}\frac{\partial z_1}{\partial b_1}$$

$$z^{(3)} = W^{(3)}h^{(2)} + b^{(3)} \qquad z^{(2)} = W^{(2)}h^{(1)} + b^{(2)} \qquad z^{(1)} = W^{(1)}x + b^{(1)}$$

$$\hat{y} = g(z^{(3)}) \qquad h^{(2)} = f^{(2)}(z^{(2)}) \qquad h^{(1)} = f^{(1)}(z^{(1)})$$

ANY QUESTION

# Optimization

- Update weights and bias $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}$ $b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial J}{\partial b^{(l)}}$
  - Batch update iterates over all training samples

**Any problems?**

# Recall: Backpropagation

- Intermediate results should be saved: z and h

$$\delta^{(l)} = \left(\boxed{W^{(l+1)T}}\delta^{(l+1)}\right) \odot \boxed{f'(z^{(l)})}$$

$$\frac{\partial J}{\partial W_3} = \boxed{\frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}}\frac{\partial z_3}{\partial W_3} \qquad \frac{\partial J}{\partial W_2} = \boxed{\frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\boxed{\frac{\partial z_3}{\partial h_2}}\boxed{\frac{\partial h_2}{\partial z_2}}}\frac{\partial z_2}{\partial W_2} \qquad \frac{\partial J}{\partial W_1} = \boxed{\frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial h_2}\frac{\partial h_2}{\partial z_2}}\frac{\partial z_2}{\partial h_1}\frac{\partial h_1}{\partial z_1}\frac{\partial z_1}{\partial W_1}$$

$$\frac{\partial J}{\partial b_3} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial b_3} \qquad \frac{\partial J}{\partial b_2} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial h_2}\frac{\partial h_2}{\partial z_2}\frac{\partial z_2}{\partial b_2} \qquad \frac{\partial J}{\partial b_1} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z_3}\frac{\partial z_3}{\partial h_2}\frac{\partial h_2}{\partial z_2}\frac{\partial h_1}{\partial z_1}\frac{\partial z_1}{\partial b_1}$$

$$z^{(3)} = W^{(3)}h^{(2)} + b^{(3)} \qquad z^{(2)} = W^{(2)}h^{(1)} + b^{(2)} \qquad z^{(1)} = W^{(1)}x + b^{(1)}$$

$$\hat{y} = g(z^{(3)}) \qquad h^{(2)} = f^{(2)}(z^{(2)}) \qquad h^{(1)} = f^{(1)}(z^{(1)})$$

# Optimization

- Update weights and bias $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}$ $b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial J}{\partial b^{(l)}}$
  - Batch update iterates over all training samples
  - Stochastic gradient descent: pick **1**-N data point at a time
    - 1: Less required memory; frequent update, converges faster

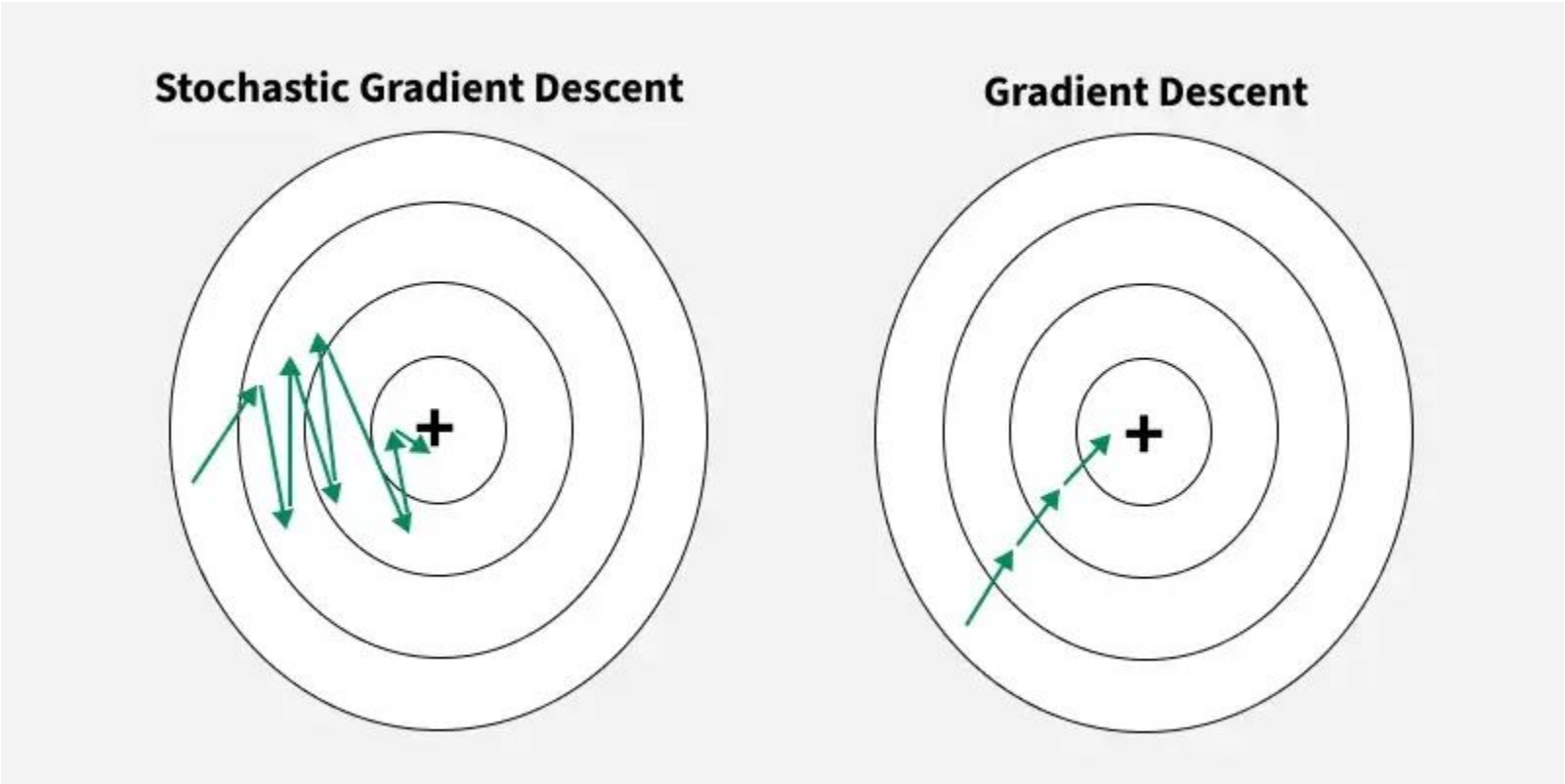
**Any problems?**

# Optimization

- Update weights and bias $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}$ $b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial J}{\partial b^{(l)}}$
  - Batch update iterates over all training samples
  - Stochastic gradient descent: pick 1-N data point at a time
    - 1: Less required memory; frequent update, converges faster
    - 1: High variance in model parameters
    - 1: Fluctuate, shoot up again after achieving global minimum

**Any solution?**

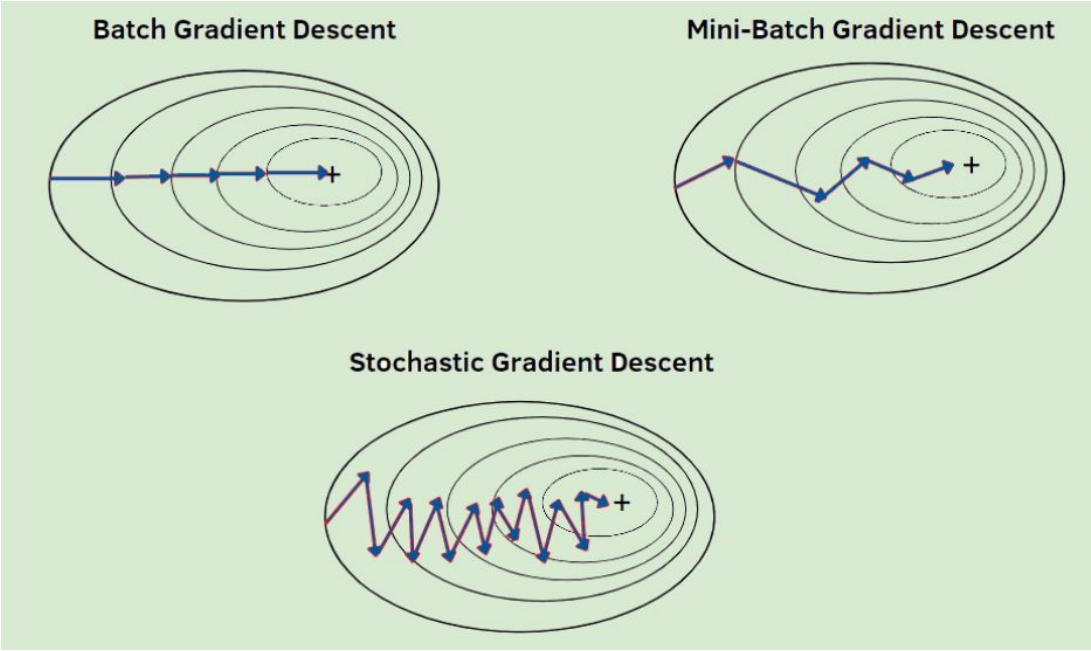
# One Examples Each Update




Image

# Optimization

- Update weights and bias $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}$ $b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial J}{\partial b^{(l)}}$
  - Batch update iterates over all training samples
  - Stochastic gradient descent: pick 1-N data point at a time
  - Mini-batches: widely used now; each time consider B samples




Image

# Optimization

- Update weights and bias $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}$ $b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial J}{\partial b^{(l)}}$
  - Batch update iterates over all training samples
  - Stochastic gradient descent: pick 1-N data point at a time
  - **<u>Mini-batches</u>**: widely used now; each time consider B samples
    - More accurate estimation of gradient
    - Smoother convergence
    - Allows for larger learning rates

    - Faster training
    - Can allow parallelization of computation

Image

# Optimizers

Stochastic Gradient Descent (SGD)

```
torch.optim.SGD(input)
```

Adam

```
torch.optim.Adam(input)
```

Adagrad

```
torch.optim.Adagrad(input)
```

Adadelta

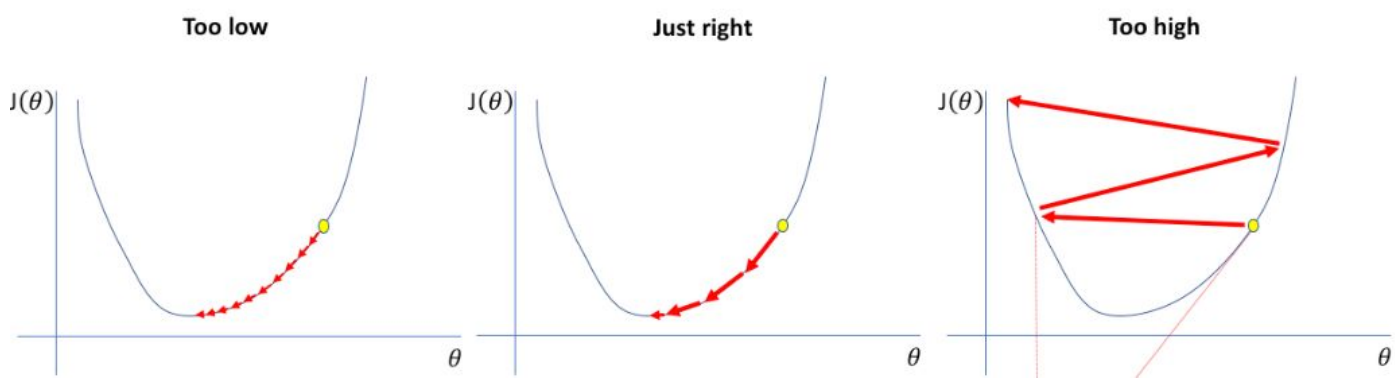
```
torch.optim.Adadelta(input)
```

RMSProp

```
torch.optim.RMSProp(input)
```

# Optimizers

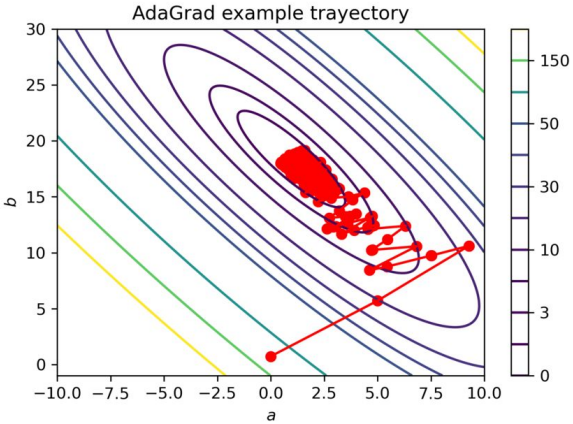
- SGD: $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}$ $b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial J}{\partial b^{(l)}}$
  - Same learning rate for all parameters and all steps
  - Sensitive to initial learning rate
    - Small: converges slowly
    - Large: fail to converge; overshoot the minimum




Image

# Optimizers

- SGD: $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}$ $b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial J}{\partial b^{(l)}}$
  - Same learning rate for all parameters and all steps
  - Sensitive to initial learning rate
- Adagrad (Adaptive Gradient) $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} g_t$
  - No manual learning rate tuning per parameter
  - Learning rate keeps shrinking ( $G_t = \sum_{i=1}^{t} g_i^2$ cumulative squared gradients, η is the initial learning rate)




Image

# Adagrad Optimizer

- Advantage:
  - This optimizer changes the learning rate
  - It changes the learning rate for each parameter and at every time step
- Disadvantage:
  - Computationally expensive as a need to calculate the square of gradients
  - The learning rate is **always decreasing** results in slow training

# Optimizers

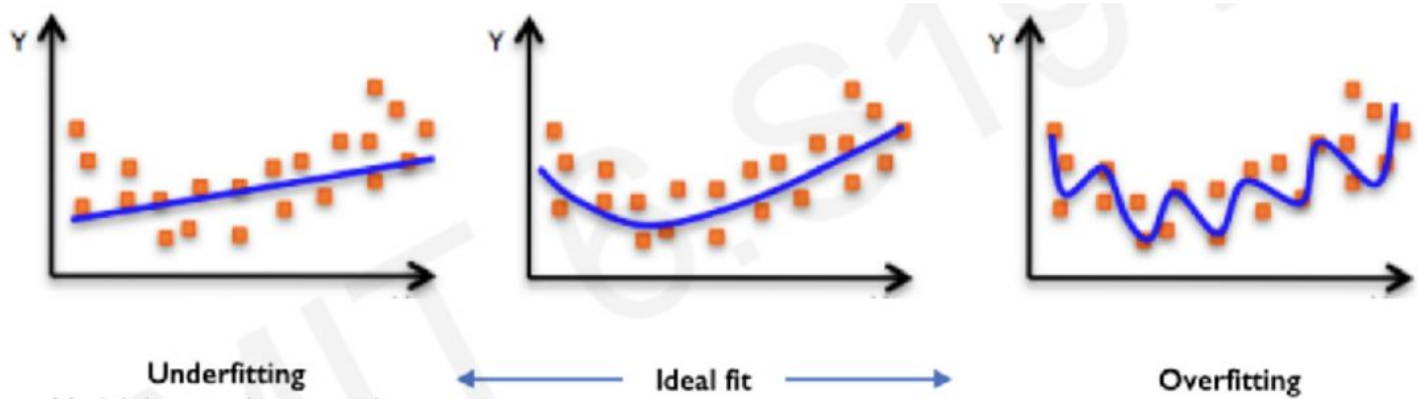
- SGD
- Adagrad (Adaptive Gradient)
- Adadelta: extension of AdaGrad
  - Remove the decaying learning Rate problem
  - limits the window of accumulated all past gradients to some fixed size window
  - Advantage:
    - The learning rate does not decay and the training does not stop.
  - Disadvantage:
    - Computationally expensive

# Optimizers

- SGD
- Adagrad (Adaptive Gradient)
- Adadelta
- **<u>Adam (Adaptive Moment Estimation)</u>**
  - High-level:
    - Large gradient history leads to smaller step
    - Small gradient history leads to larger step
    - <u>momentum</u> + adaptive scaling (Adadelta)
      - Smooth direction
  - Fast convergence, minimal tuning, widely used
  - More memory, may converge to sharp minima

# Issue during Optimization

- Underfitting vs. Overfitting
  - Underfitting: does not learn the data well
    - Low training error; high validation/test error
  - Overfitting: to complex networks, does not generalize well



# Solutions

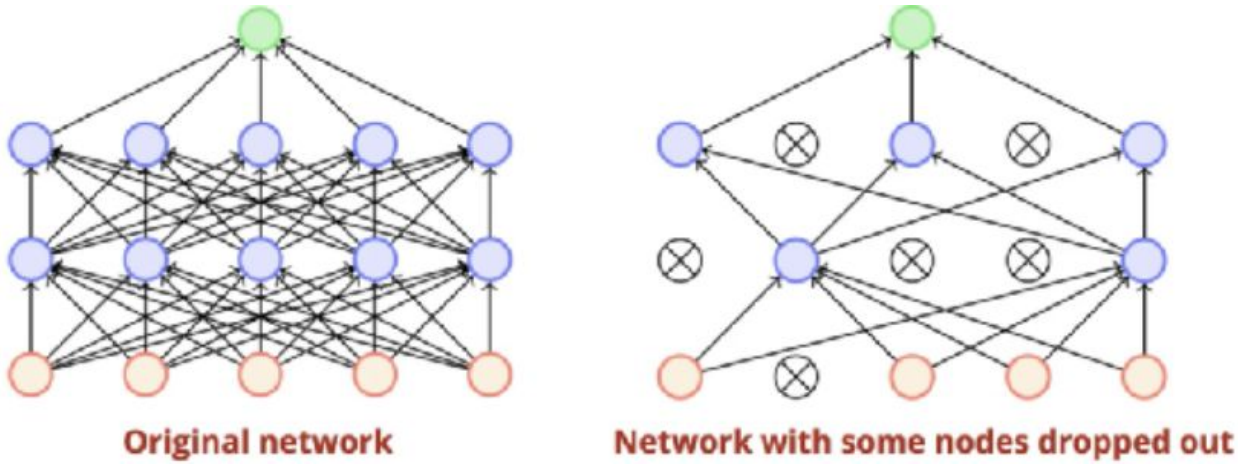
- Model design
  - increase/reduce model capacity: number of layers, neurons

# Solutions

- Model design
- Regularization:
  - Dropout
  - Early stopping
  - Data augmentation
  - L1 norms, L2 norms, batch normalization

# Dropout

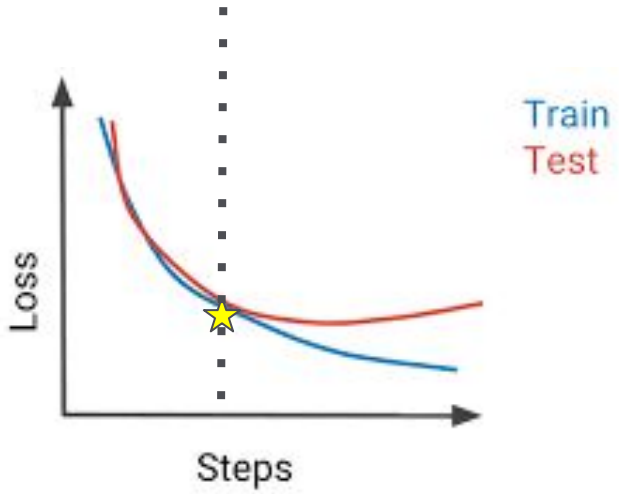
- During training, randomly set some activations to zero
  - Forces the network not to rely on any 1 node




```
self.drop_layer = torch.nn.Dropout(p=0.5)
```

# Early Stopping

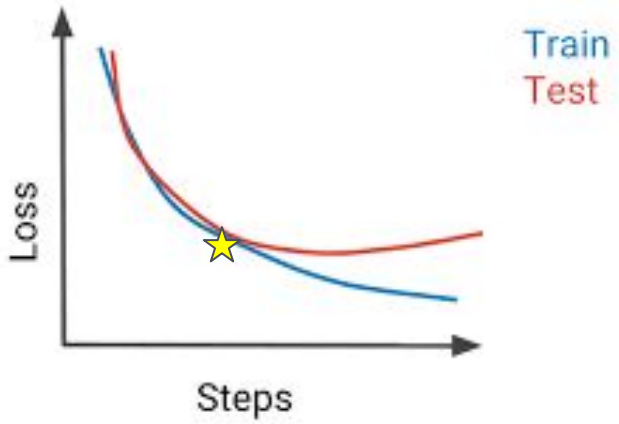
- Stop training before we have a chance to overfit
  - Left: underfitting
  - Right: overfitting




image

# Early Stopping

- Stop training before we have a chance to overfit




image

# Data Augmentation in CV

- Transform each training image into multiple slightly different versions
  - Prohibit memorization of training data
  - E.g., image classification: rotation, gaussian noise, crop

# Data Augmentation in CV

- Transform each training image into multiple slightly different versions
  - Prohibit memorization of training data
  - E.g., image classification: rotation, gaussian noise, crop

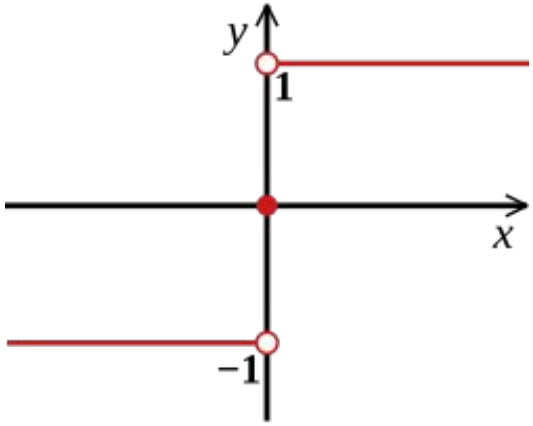
```
final_train_data = []
final_target_train = []
for i in tqdm(range(train_x.shape[0])):
    final_train_data.append(train_x[i])
    final_train_data.append(rotate(train_x[i], angle=45, mode = 'wrap'))
    final_train_data.append(np.fliplr(train_x[i]))
    final_train_data.append(np.flipud(train_x[i]))
    final_train_data.append(random_noise(train_x[i],var=0.2**2))
    for j in range(5):
        final_target_train.append(train_y[i])
```

# Data Augmentation in NLP

- Word-level
  - Synonym replacement: it was great → it was fantastic
  - Random insertion, deletion, swap
- Sentence -level:
  - Back translation: English → Chinese → English
  - Rephrasing: with the help of LLMs

# L1 & L2 Norms

- Add regularizers to loss
  - L1: $J_{new} = J(w) + \lambda \sum_i |w_i|$
  - λ controls regularization strength
  - L1 norm: produces sparse weights $w_i \leftarrow w_i - \eta \left( \frac{\partial L}{\partial w_i} + \lambda \, \mathrm{sign}(w_i) \right)$
    - Push value towards 0
    - weight>0: $w_i \leftarrow w_i - \eta(\text{data gradient} + \lambda)$
    - weight<0: $w_i \leftarrow w_i - \eta(\text{data gradient} - \lambda)$
      - Like adding λ, pushing towards 0



# L1 Norm

- Updated loss: $J_{new} = J(w) + \lambda \sum_i |w_i|$
  - Feature selection by assigning insignificant input features with zero weight and useful features with a non zero weight.
  - λ how much to penalizes the weights
    - λ =0? Original loss
    - λ is large: very simple model having a high bias or is underfitting

# L1 & L2 Norms

- Add regularizers to loss
  - L2: $J_{new} = J(w) + \lambda \sum_i w_i^2$
  - λ controls regularization strength $w_i \leftarrow w_i - \eta(2\lambda w_i)$
  - L2 norm: smooth small weights (omit original gradients)
    - Gradually shrink weights
    - Large weights shrink more
    - Small weights shrink less
    - But not exactly zero

# L2 Norm

- Updated loss: $J_{new} = J(w) + \lambda \sum_i w_i^2$
  - Forces the weights to be small but does not make them zero
  - Performs better when all the input features influence the output and all with weights are of roughly equal size
  - Not robust to outliers as square terms blows up the error differences of the outliers
    - The regularization term tries to fix it by penalizing the weights

# L1 & L2 Norm

- L2 can be embedded in the optimizer

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4, weight_decay=1e-5)
```

- Or explicitly add L1/L2 normalization term

```
lambda1, lambda2 = 0.5, 0.01

optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)

optimizer.zero_grad()
outputs, layer1_out, layer2_out = model(inputs)
cross_entropy_loss = F.cross_entropy(outputs, targets)

all_linear1_params = torch.cat([x.view(-1) for x in model.linear1.parameters()])
all_linear2_params = torch.cat([x.view(-1) for x in model.linear2.parameters()])
l1_regularization = lambda1 * torch.norm(all_linear1_params, 1)
l2_regularization = lambda2 * torch.norm(all_linear2_params, 2)

loss = cross_entropy_loss + l1_regularization + l2_regularization
loss.backward()
optimizer.step()
```

# Batch Normalization

- Compute batch mean, variance

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

- Normalize batch

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

- Scale and shift $y_i = \gamma \hat{x}_i + \beta$
- Stabilize training and speeds up convergence
- Improve accuracy

ANY QUESTION

# Hyperparameters

- What we have seen
  - Learning rate
  - Batch size in mini-batch training
  - Training iterations: when to stop training
  - Dimension of hidden units
  - Dropout rate
  - ……

# Hyperparameters

- Hyperparameters:
  - Learning rate
  - Batch size in mini-batch training
  - Training iterations: when to stop training
  - Dimension of hidden units
  - Dropout rate
  - ……
- Hyperparameters determine convergence speed, stability and final performance!

# Hyperparameters

- How to choose your hyperparameters:
  - **<u>Minimize validation errors</u>**
- Different splits:
  - Training data: learn parameters
  - Validation data: choose hyperparameters
  - Test data: final performance evaluation (unseen)

# Example: Leaderboard Competition

- Public leaderboard: train/dev split
  - Ground-truth labels provided
- Private leaderboard: test split
  - Only input data provided
  - Labels are hidden

# Tuning Hyperparameters

- Manual Search: simple but hard to scale
  - Rely on experience + trial and error
- Grid Search: exhaustive but expensive
  - Try combinations of hyperparameters
  - E.g., learning rage {1e−5,1e−4}, batch size: {32,64,128,256}, dropout rate: {0.2,0.5, 0.7}
  - **How many combinations?**

# Tuning Hyperparameters

- Manual Search: simple but hard to scale
  - Rely on experience + trial and error
- Grid Search: exhaustive but expensive
  - Try combinations of hyperparameters
- Random Search: still brute force
  - Sample hyperparameters from distributions

# Importance of Hyperparameters

- 2017 CVPR Tips and Tricks for Visual Question Answering

| | VQA v2 validation | | | | |
|---|---|---|---|---|---|
| | VQA Score | | | | Accuracy over |
| | All | Yes/no | Numbers | Other | balanced pairs |
| **Reference model** | 63.15 ±0.08 | 80.07 | 42.87 | 55.81 | 34.66 |
| **Training data** (reference: with Visual Genome data; shuffling keeps balanced pairs in the same minibatches) | | | | | |
| Without extra training data from Visual Genome | 62.48 ±0.15 | 80.37 | 42.06 | 54.44 | 34.12 |
| Random shuffling of training data | 63.16 ±0.06 | 80.17 | 42.92 | 55.73 | 34.54 |
| Discard training questions without answers of score=1.0 | 63.15 ±0.12 | 80.15 | 42.90 | 55.73 | 34.62 |
| **Question embedding** (reference: 300-dimensional GloVe word embeddings, 1-layer forward GRU) | | | | | |
| Word embeddings learned from random initialization; 1-layer forward GRU | 62.28 ±0.06 | 78.77 | 41.92 | 55.27 | 33.67 |
| 100-dimensional GloVe; forward GRU | 62.45 ±0.05 | 78.84 | 42.12 | 55.51 | 33.70 |
| 200-dimensional GloVe; forward GRU | 62.96 ±0.12 | 79.76 | 42.49 | 55.75 | 34.34 |
| 300-dimensional GloVe; bag-of-words (sum) | 62.14 ±0.04 | 79.54 | 41.88 | 54.41 | 33.73 |
| 300-dimensional GloVe; bag-of-words (average) | 62.53 ±0.09 | 80.17 | 42.15 | 54.67 | 34.22 |
| 300-dimensional GloVe; 1-layer backward GRU | 62.82 ±0.02 | 79.57 | 42.39 | 55.64 | 33.98 |
| 300-dimensional GloVe; 2-layer forward GRU | 62.29 ±0.10 | 78.66 | 42.42 | 55.23 | 33.56 |
| 300-dimensional GloVe randomly shuffled; 1-layer forward GRU | 62.16 ±0.08 | 78.74 | 41.73 | 55.11 | 33.45 |
| **Image features** (reference: image features from bottom-up attention, adaptive $K$) | | | | | |
| ResNet-200 global features ($K$=1, *i.e.* without image attention)) | 56.16 ±0.14 | 75.89 | 36.39 | 46.57 | 25.93 |
| ResNet-200 features 14×14 ($K$=196)) | 57.93 ±0.25 | 76.17 | 36.25 | 49.95 | 27.64 |
| ResNet-200 features downsampled to 7×7 ($K$=49) | 59.35 ±0.10 | 77.34 | 37.74 | 51.55 | 29.49 |
| Image features from bottom-up attention, $K$=36 | 62.82 ±0.14 | 79.92 | 42.44 | 55.35 | 34.18 |
| **Image attention** (reference: image features from bottom-up attention, 1 attention head, softmax normalization) | | | | | |
| ResNet-200 7×7 features, 1 head, sigmoid normalization | 58.96 ±0.37 | 77.05 | 37.99 | 50.90 | 28.68 |
| ResNet-200 7×7 features, 1 head, softmax normalization | 59.35 ±0.10 | 77.34 | 37.74 | 51.55 | 29.49 |
| ResNet-200 7×7 features, 2 heads, sigmoid normalization | 58.70 ±0.30 | 76.42 | 37.95 | 50.87 | 28.40 |
| ResNet-200 7×7 features, 2 heads, softmax normalization | 59.20 ±0.13 | 76.97 | 37.52 | 51.58 | 29.43 |
| Image features from bottom-up attention, 1 head, sigmoid normalization | 62.15 ±1.41 | 78.83 | 43.44 | 54.57 | 32.98 |
| Image features from bottom-up attention, 2 heads, sigmoid normalization | 62.91 ±0.26 | 79.74 | 43.99 | 55.27 | 34.30 |
| Image features from bottom-up attention, 2 heads, softmax normalization | 63.10 ±0.05 | 79.80 | 43.17 | 55.82 | 34.52 |
| **Output vocabulary** (reference: >8 occurrences as correct answers in the training data, $N = 3,129$ | | | | | |
| Keep answers with >6 training occurrences, $N = 3,793$ | 63.26 ±0.07 | 80.23 | 42.94 | 55.89 | 34.50 |
| Keep answers with >10 training occurrences, $N = 2,748$ * | 63.30 ±0.06 | 80.33 | 43.02 | 55.86 | 34.69 |
| Keep answers with >12 training occurrences, $N = 2,418$ | 63.16 ±0.08 | 80.18 | 43.17 | 55.66 | 34.56 |
| Keep answers with >14 training occurrences, $N = 2,160$ | 63.27 ±0.08 | 80.45 | 43.07 | 55.69 | 34.83 |
| Keep answers with >16 training occurrences, $N = 1,961$ | 63.17 ±0.10 | 80.29 | 42.94 | 55.65 | 34.62 |
| Keep answers with >20 training occurrences, $N = 1,656$ | 63.14 ±0.05 | 80.21 | 42.90 | 55.66 | 34.67 |
| Keep answers with >200 training occurrences, $N = 278$ | 59.59 ±0.04 | 79.90 | 42.11 | 48.93 | 32.64 |
| **Output classifier** (reference: sigmoid output, $\boldsymbol{w}_o^{\text{text}}$ and $\boldsymbol{w}_o^{\text{img}}$ pretrained | | | | | |
| Softmax output, $\boldsymbol{w}_o^{\text{text}}$ pretrained, $\boldsymbol{w}_o^{\text{img}}$ pretrained | 60.47 ±0.08 | 78.19 | 40.93 | 52.32 | 32.27 |
| Sigmoid output, $\boldsymbol{w}_o^{\text{text}}$ randomly initialized, $\boldsymbol{w}_o^{\text{img}}$ randomly initialized | 62.28 ±0.99 | 78.50 | 42.50 | 55.32 | 33.14 |
| Sigmoid output, $\boldsymbol{w}_o^{\text{text}}$ randomly shuffled, $\boldsymbol{w}_o^{\text{img}}$ randomly shuffled | 62.64 ±0.15 | 79.82 | 42.26 | 55.13 | 34.17 |
| Sigmoid output, $\boldsymbol{w}_o^{\text{text}}$ pretrained, $\boldsymbol{w}_o^{\text{img}}$ randomly initialized | 62.94 ±0.14 | 79.99 | 42.72 | 55.47 | 34.26 |
| Sigmoid output, $\boldsymbol{w}_o^{\text{text}}$ pretrained, $\boldsymbol{w}_o^{\text{img}}$ randomly shuffled | 63.04 ±0.02 | 80.20 | 42.83 | 55.50 | 34.66 |
| Sigmoid output, $\boldsymbol{w}_o^{\text{text}}$ randomly initialized, $\boldsymbol{w}_o^{\text{img}}$ pretrained | 63.21 ±0.01 | 80.06 | 42.97 | 55.90 | 34.65 |
| Sigmoid output, $\boldsymbol{w}_o^{\text{text}}$ randomly shuffled, $\boldsymbol{w}_o^{\text{img}}$ pretrained | 62.97 ±0.06 | 79.83 | 42.62 | 55.68 | 34.31 |
| **General architecture** (reference: gated tanh non-linear activations, hidden states of dimension 512) | | | | | |
| Non-linear activations: ReLU | 61.63 ±0.21 | 78.48 | 41.15 | 54.39 | 32.57 |
| Non-linear activations: tanh | 61.74 ±0.44 | 79.20 | 40.92 | 54.13 | 33.10 |
| Non-linear activations: gated ReLU | 62.44 ±0.07 | 79.33 | 42.12 | 55.13 | 33.52 |
| Hidden states of dimension 256 | 62.80 ±0.05 | 79.62 | 42.46 | 55.53 | 34.12 |
| Hidden states of dimension 384 | 63.12 ±0.11 | 80.06 | 42.89 | 55.74 | 34.54 |
| Hidden states of dimension 768 | 63.12 ±0.11 | 80.06 | 42.89 | 55.74 | 34.54 |
| Hidden states of dimension 1024 | 63.02 ±0.55 | 79.88 | 42.68 | 55.73 | 34.34 |
| Hidden states of dimension 1280 * | 63.37 ±0.21 | 80.40 | 43.02 | 55.96 | 34.76 |
| **Mini-batch size** (reference: 512 training questions) | | | | | |
| 128 Training questions | 62.38 ±0.08 | 79.70 | 42.29 | 54.67 | 34.01 |
| 256 Training questions * | 63.17 ±0.09 | 80.22 | 42.94 | 55.72 | 34.71 |
| 384 Training questions * | 63.20 ±0.04 | 80.21 | 43.08 | 55.75 | 34.61 |
| 768 Training questions | 62.99 ±0.12 | 79.84 | 42.61 | 55.73 | 34.40 |

Table 2. Ablations of a single network, evaluated on the VQA v2 validation set. We evaluate variations of our best "reference"

# Importance of Hyperparameters

- 2017 CVPR Tips and Tricks for Visual Question Answering
  - Performance is very dependent on design choices as well as **various details of the implementation!**

ANY QUESTION